

ĐẠI HỌC QUỐC GIA TP.HCM  
TRƯỜNG ĐẠI HỌC BÁCH KHOA

PHẠM THẾ VINH

THIẾT KẾ CẤU TRÚC PHẦN CỨNG CHO MẠNG  
NƠON TÍCH CHẬP

Chuyên ngành: Kỹ thuật Điện Tử

Mã số: 60520203

LUẬN VĂN THẠC SĨ

TP. HỒ CHÍ MINH, THÁNG 01 NĂM 2020

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI:

**Trường Đại học Bách Khoa – ĐHQG-HCM**

Cán bộ hướng dẫn khoa học: Tiến Sĩ Trương Quang Vinh.

Cán bộ chấm nhận xét 1: .....

Cán bộ chấm nhận xét 2: .....

Luận văn thạc sĩ được bảo vệ tại Trường Đại học Bách Khoa, ĐHQG Tp. HCM ngày 09 tháng 01 năm 2020.

Thành phần Hội đồng đánh giá luận văn thạc sĩ gồm:

1. Chủ Tịch: Phó Giáo Sư, Tiến Sĩ Hà Hoàng Kha.
2. Phản Biện 1: Tiến Sĩ Bùi Trọng Tú.
3. Phản Biện 2: Tiến Sĩ Nguyễn Minh Sơn.
4. Ủy Viên: Tiến Sĩ Trần Hoàng Linh.
5. Thư Ký: Tiến Sĩ Nguyễn Lý Thiên Trường.

Xác nhận của Chủ tịch Hội đồng đánh giá LV và Trưởng Khoa quản lý chuyên ngành sau khi luận văn đã được sửa chữa (nếu có).

**CHỦ TỊCH HỘI ĐỒNG**

**TRƯỞNG KHOA.....**

**NHIỆM VỤ LUẬN VĂN THẠC SĨ**

Họ tên học viên: Phạm Thế Vinh ..... MSHV: 1670849 .....

Ngày, tháng, năm sinh: 07/04/1994..... Nơi sinh: TP.Hồ Chí Minh

Chuyên ngành: Kỹ thuật Điện Tử..... Mã số: 60520203 .....

**I. TÊN ĐỀ TÀI:** THIẾT KẾ CẤU TRÚC PHẦN CỨNG CHO MẠNG NƠ-  
RON TÍCH CHẬP .....

**NHIỆM VỤ VÀ NỘI DUNG:**

- Tìm hiểu giải thuật CNN, mô hình VGG16.
- Thiết kế kiến trúc phần cứng cho CNN.
- Hiện thực cấu trúc CNN trên FPGA.
- Thực hiện ứng dụng cấu trúc CNN trên FPGA để nhận dạng cảm xúc khuôn mặt.

**II. NGÀY GIAO NHIỆM VỤ:** (Ghi theo trong QĐ giao đề tài): 11/02/2019....

**III. NGÀY HOÀN THÀNH NHIỆM VỤ:** (Ghi theo trong QĐ giao đề tài):  
08/12/2019 .....

**IV. CÁN BỘ HƯỚNG DẪN** (Ghi rõ học hàm, học vị, họ, tên):

Tiến Sĩ Trương Quang Vinh.....

.....

*Tp. HCM, ngày ... tháng ... năm 20...*

**CÁN BỘ HƯỚNG DẪN**

(Họ tên và chữ ký)

**CHỦ NHIỆM BỘ MÔN ĐÀO TẠO**

(Họ tên và chữ ký)

**TRƯỞNG KHOA.....**

(Họ tên và chữ ký)

## LỜI CẢM ƠN

Em xin được gửi lời cảm ơn chân thành đến thầy Trương Quang Vinh đã hướng dẫn và giúp đỡ em trong suốt quá trình thực hiện đề tài. Đồng thời gửi lời cảm ơn đến đồng sự Đinh Việt Hải đã đồng hành trong quá trình nghiên cứu.

Tp Hồ Chí Minh, tháng 01 năm 2020

Phạm Thế Vinh

## TÓM TẮT LUẬN VĂN

Mạng nơon tích chập ( Convolutional Neural Network hay CNN ) là một trong những thuật toán mạnh nhất hiện nay cho bài toán phân loại. Tuy nhiên, những mạng như VGG đòi hỏi rất nhiều sức mạnh tính toán ở các lớp tích chập. Do đó các nhà nghiên cứu đã tạo ra nhiều bộ tăng tốc đã được thiết kế nhằm cải thiện tốc độ tính toán của mạng, tuy nhiên hầu hết đều sử dụng những phần cứng FPGA mạnh với số lượng tài nguyên phong phú tuy nhiên chi phí cao. Trong nghiên cứu này tôi đề xuất một bộ tăng tốc cho mạng CNN mà cụ thể là mạng VGG16 trên phần cứng FPGA chi phí thấp với số lượng tài nguyên bị giới hạn. Do đặc tính giống nhau giữa các lớp mạng trong VGG16, tôi thiết kế một bộ tăng tốc chuyên dụng với các tham số có thể tái thiết lập bằng phần mềm. Tôi tiến hành đánh giá bộ tăng tốc đã thiết kế với mạng VGG16 và đạt tốc độ 38.8 GOP/s ở tốc độ xung clock 150Mhz trên chip Cyclone V SoC. Sau đó, để đưa lõi tăng tốc vào ứng dụng thực tế, tôi tiến hành sử dụng lõi cho một mạng mà tôi tự xây dựng nhằm phân loại cảm xúc khuôn mặt. Mục tiêu của việc xây dựng mô hình ứng dụng này là thiết kế một hệ thống phân loại độc lập có khả năng tự vận hành với chi phí thấp và tiêu hao ít năng lượng.

## ABSTRACT

Convolutional Neural Network (CNN) is one of the most powerful algorithms available today. However, networks like VGG require a lot of computational power in convolution layers. So researchers have created many accelerators that are designed to improve the computing speed of the network, but most use powerful FPGA hardware with lots of resources, but costly. In this study, We proposed an accelerator for CNN, specifically VGG16 on low-cost FPGA hardware, with a limited number of resources. Due to the similarity between the network layers in VGG16, We designed a dedicated accelerator with parameters that can be reconfigured by software. I evaluated the accelerator designed with the VGG16 network and reached 38.8 GOP / s at a clock speed of 150 MHz on the Cyclone V SoC chip. Then, to get the core into practical applications, We used the core for a neural network, which built to classify facial emotions. The goal of building this application model is to design an independent classification system that is able to operate on its own with low cost and low energy consumption.

## **LỜI CAM ĐOAN**

Tôi tên Phạm Thế Vinh, là học viên cao học ngành Kỹ thuật Điện Tử tại trường Đại học Bách Khoa, thành phố Hồ Chí Minh. Tôi xin cam đoan công trình nghiên cứu này là do chính tôi thực hiện. Trong suốt quá trình thực hiện đề tài này, các tư liệu tham khảo sử dụng được trích dẫn từ các nguồn thực tế, uy tín và chất lượng. Kết quả thu được được thực hiện một cách độc lập và hoàn toàn trung thực.

Tp. Hồ Chí Minh, tháng 01 năm 2020

Phạm Thế Vinh

## MỤC LỤC

|                                                                                              |     |
|----------------------------------------------------------------------------------------------|-----|
| LỜI CẢM ƠN .....                                                                             | i   |
| TÓM TẮT LUẬN VĂN .....                                                                       | ii  |
| ABSTRACT .....                                                                               | iii |
| LỜI CAM ĐOAN .....                                                                           | iv  |
| MỤC LỤC .....                                                                                | v   |
| DANH SÁCH HÌNH VẼ .....                                                                      | vii |
| DANH SÁCH BẢNG BIỂU .....                                                                    | ix  |
| CHƯƠNG 1: MỞ ĐẦU .....                                                                       | 2   |
| 1.1. Lý Do Chọn Đề Tài: .....                                                                | 2   |
| 1.2. Mục Tiêu Và Nhiệm Vụ Của Luận Văn: .....                                                | 3   |
| CHƯƠNG 2: TỔNG QUAN .....                                                                    | 4   |
| 2.1 Mạng VGG16: .....                                                                        | 4   |
| 2.2 Các Kỹ Thuật Tối Ưu Vòng Lặp Trong Mạng CNN: .....                                       | 5   |
| 2.3 DE10 Standard Board: .....                                                               | 6   |
| CHƯƠNG 3: THIẾT KẾ HỆ THỐNG .....                                                            | 8   |
| 3.1 Kiến Trúc Chung: .....                                                                   | 8   |
| 3.2 Sơ Đồ Thiết Kế Lỗi IP CNN Trong Hệ Thống Soc Cyclone V: .....                            | 9   |
| 3.3 Input Buffer (IB): .....                                                                 | 11  |
| 3.4 Processing Engine (PE): .....                                                            | 15  |
| 3.5 Output Buffer (OB): .....                                                                | 18  |
| 3.6 Register: .....                                                                          | 18  |
| 3.7 Giao Tiếp Avalon MM: .....                                                               | 20  |
| 3.8 Memory: .....                                                                            | 20  |
| 3.8.1 External Mem: .....                                                                    | 20  |
| 3.8.2 On-Chip Memory: .....                                                                  | 21  |
| 3.8.3 Input Feature Map Memory: .....                                                        | 21  |
| 3.8.4 Weights Memory: .....                                                                  | 22  |
| 3.8.5 Bias Memory: .....                                                                     | 23  |
| CHƯƠNG 4: NGUYÊN LÝ HOẠT ĐỘNG .....                                                          | 24  |
| 4.1 Nguyên Lý Hoạt Động Của Lỗi Tăng Tốc: .....                                              | 24  |
| 4.2 Phần Mềm Trên Lỗi Arm: .....                                                             | 27  |
| CHƯƠNG 5: KẾT QUẢ THỰC HIỆN .....                                                            | 29  |
| 5.1 Cài Đặt Hệ Thống Và Hiện Thực: .....                                                     | 29  |
| 5.2 So sánh: .....                                                                           | 30  |
| CHƯƠNG 6: SỬ DỤNG LỖI TĂNG TỐC ĐÃ THIẾT KẾ CHO ỨNG DỤNG<br>PHÂN LOẠI CẢM XÚC KHUÔN MẶT ..... | 32  |
| 6.1 Nhận Diện Cảm Xúc Khuôn Mặt: .....                                                       | 32  |
| 6.1.1 Bước Phát Hiện Khuôn Mặt: .....                                                        | 32  |
| 6.1.2 Bước Phân Loại Cảm Xúc Khuôn Mặt: .....                                                | 33  |

|                                     |    |
|-------------------------------------|----|
| 6.2 Thiết Kế Hệ Thống:.....         | 34 |
| 6.2.1 Tổng Quan Kiến Trúc:.....     | 34 |
| 6.2.2 Kiến Trúc HPS: .....          | 35 |
| 6.2.3 Kiến trúc lõi tăng tốc: ..... | 35 |
| 6.3 Thử Nghiệm Và Kết Quả: .....    | 36 |
| CHƯƠNG 7: KẾT LUẬN .....            | 41 |
| TÀI LIỆU THAM KHẢO.....             | 42 |
| CHƯƠNG 8: PHỤ LỤC .....             | 44 |

## DANH SÁCH HÌNH VẼ

|                                                                                             |    |
|---------------------------------------------------------------------------------------------|----|
| Hình 2.1 VGG like neural network.                                                           | 4  |
| Hình 2.2 Bốn loại vòng lặp trong CNN.                                                       | 6  |
| Hình 2.3 Cấu trúc chung của hệ thống Cyclone V SoC.                                         | 7  |
| Hình 3.1 Kiến trúc chung của bộ tăng tốc VGG.                                               | 8  |
| Hình 3.2 Trình tự hoạt động của vi điều khiển và bộ tăng tốc.                               | 9  |
| Hình 3.3 Chi tiết kiến trúc hệ thống.                                                       | 9  |
| Hình 3.4 Sơ đồ Top-level lõi CNN.                                                           | 10 |
| Hình 3.5 Sơ đồ khối các module của lõi CNN.                                                 | 11 |
| Hình 3.6 Thiết kế Input Buffer.                                                             | 11 |
| Hình 3.7 Quá trình thực hiện tích chập giữa trọng số và box (input window) từ các kênh màu. | 13 |
| Hình 3.8 Sơ đồ khối Input Buffer.                                                           | 13 |
| Hình 3.9 Máy trạng thái Input Buffer.                                                       | 14 |
| Hình 3.10 Mô tả và phân loại các vòng lặp trong lớp chập.                                   | 15 |
| Hình 3.11 Processing Engine cho VGG16.                                                      | 16 |
| Hình 3.12 Sơ đồ khối dữ liệu vào Process Engine và các Processing Unit.                     | 17 |
| Hình 3.13 Process Engine với Relu và Max Pooling.                                           | 17 |
| Hình 3.14 Sơ đồ khối Output Buffer.                                                         | 18 |
| Hình 3.15 Phân tầng vùng nhớ.                                                               | 20 |
| Hình 3.16 Tổ chức vùng nhớ Shared memory.                                                   | 21 |
| Hình 3.17 Cách sắp xếp dữ liệu input feature map vào imem.                                  | 22 |
| Hình 3.18 Cách sắp xếp dữ liệu trọng số vào bộ nhớ wmem.                                    | 22 |
| Hình 3.19 Tổ chức vùng nhớ chữ bias.                                                        | 23 |
| Hình 4.1 Cửa sổ trượt đồng thời trên 2 imem 1 lúc.                                          | 24 |
| Hình 4.2 Lần trượt thứ 2 sau khi đã đi hết 2 imem 0 và 1.                                   | 24 |
| Hình 4.3 Dữ liệu từ imem được đưa vào PU.                                                   | 25 |
| Hình 4.4 Diagram nguyên lý của Data router.                                                 | 25 |

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| Hình 4.5 cửa sổ trượt box màu xanh duy chuyển dọc theo vùng nhớ là box màu đỏ. | 26 |
| Hình 4.6 Diagram ngõ vào của các PE.                                           | 26 |
| Hình 4.7 Vai trò của các thành phần firmware trên ARM.                         | 27 |
| Hình 4.8 Flowchart xử lý các lớp tích chập.                                    | 28 |
| Hình 5.1 lõi IP đã thiết kế trong Qsys.                                        | 30 |
| Hình 5.2 Bảng thiết lập các tham số cho lõi IP.                                | 30 |
| Hình 6.1 Kiến trúc mạng CNN được thiết kế.                                     | 34 |
| Hình 6.2 Kiến trúc không đồng nhất giữa CPU và FPGA.                           | 35 |
| Hình 6.3 Kiến trúc bộ tăng tốc.                                                | 36 |
| Hình 6.4 Phân bố hình ảnh trong các lớp.                                       | 37 |
| Hình 6.5 Trái: độ chính xác của mô hình, phải: loss của mô hình.               | 37 |
| Hình 6.6 Confusion matrix khi thử nghiệm trên tập dữ liệu FER2013.             | 38 |
| Hình 6.7 Hình ảnh demo hệ thống.                                               | 39 |
| Hình 7.1 Cấu trúc firmware trên ARM.                                           | 44 |

## **DANH SÁCH BẢNG BIỂU**

|                                                                |    |
|----------------------------------------------------------------|----|
| Bảng 2.1 Các thiết lập cho các lớp khác nhau của mạng VGG16.   | 5  |
| Bảng 2.2 Tài nguyên có sẵn trên chip Intel SoC FPGA Cyclone V. | 7  |
| Bảng 3.1 Chi tiết các thanh ghi điều khiển.                    | 19 |
| Bảng 5.1 Tài nguyên đã sử dụng.                                | 29 |
| Bảng 5.2 So sánh với những nghiên cứu khác.                    | 30 |
| Bảng 6.1 Tài nguyên đã sử dụng trong thiết kế.                 | 38 |
| Bảng 6.2 So sánh với nghiên cứu khác.                          | 39 |

## CHƯƠNG 1: MỞ ĐẦU

### 1.1. Lý Do Chọn Đề Tài:

Mạng nơron tích chập đã phát triển nhanh chóng và trở thành một công cụ quan trọng trong ứng dụng phân loại, đặc biệt đối với các dữ liệu hình ảnh và âm thanh. Trong những năm gần đây, nhiều mạng CNN mạnh mẽ đã được đề xuất nhằm cải thiện độ chính xác như Alexnet [1], VGG [2], Inception [3],... các phương pháp dựa trên mạng VGG là một hướng tiếp cận phổ biến trong các ứng dụng như: phân loại cây [4], phân loại hoa [5], phân loại cảm xúc khuôn mặt [6],... Tuy nhiên, mạng VGG đòi hỏi rất nhiều khả năng tính toán, cần đến 38 GOP để xử lý chỉ 1 ảnh 244x244 RGB. Điều này dẫn đến độ trễ lớn khi hiện thực mạng trên phần mềm. Nhiều phần cứng đã được đề xuất sử dụng dựa trên GPU, FPGA, ASIC [7-10] nhằm giải quyết vấn đề về khả năng tính toán. Trong đó, các nền tảng tăng tốc dựa trên FPGA ưu thế hơn so với các nền tảng khác vì lợi thế có thể tái lập trình và hiệu quả về mặt năng lượng.

Thực tế cho thấy hơn 90% các phép tính của CNN nằm ở lớp tích chập [9] trong khi lớp fully-connected lại tiêu tốn nhiều bộ nhớ. Lấy ví dụ, VGG16 cần 30.69 GOP cho tất cả 13 lớp tích chập, do đó cần đến hơn 400MB không gian bộ nhớ để lưu tất cả các tham số (ở dạng số float). Điều đó dẫn tới việc các tác giả tập trung cải thiện khả năng tính toán bằng việc áp dụng các kỹ thuật như Winograd[11], FFT[12], loop optimization [8-9], stochastic [13], ... Những nghiên cứu này đạt được hiệu năng rất cao nhưng không làm giảm đi độ chính xác của mạng. Bên cạnh tính hiệu quả của thuật toán thì việc sử dụng chip FPGA cao cấp và mạnh cũng là một lý do quan trọng trong kết quả đạt được. Do đó, bài toán trở nên phức tạp hơn khi hiện thực một mạng có kích thước lớn như VGG xuống chip FPGA chi phí thấp.

Trong nghiên cứu này, tôi đề xuất một bộ tăng tốc có thể được tái thiết lập cấu hình hoạt động và tương thích với các lớp tích chập của mạng VGG và có thể chạy trên chip FPGA chi phí thấp là Intel Cyclone V SoC. Bộ tăng tốc này nằm trong một hệ thống không đồng nhất (heterogeneous system), hệ thống này bao gồm một vi xử lý ARM và thành phần FPGA. Trong khi bộ vi điều khiển có trách nhiệm thiết lập

các tham số cho từng lớp tích chập và chuẩn bị các dữ liệu cho phần FPGA. Thành phần FPGA thực hiện tất cả các hoạt động còn lại như là: zero padding, tích chập, max pooling và activation. Hơn nữa, bằng việc tối ưu multi-port của bộ điều khiển SDRAM, tôi đạt được băng thông bộ nhớ cao hơn, lên tới 4.6GB/s. Do đó, hệ thống của tôi đạt được hiệu năng là 38.8 GOP/s cho các lớp tích chập của mạng VGG16 với tần số hoạt động là 150MHz cũng như sử dụng tài nguyên hệ thống hiệu quả hơn so với các nghiên cứu trước đây.

Phần tiếp theo của luận văn, trong chương 2 tôi sẽ trình bày tổng quan mạng nơron tích chập, mạng VGG, các kỹ thuật tối ưu vòng lặp và giới thiệu về board DE10-Standard với chip Cyclone V của Intel bên trong. Tôi mô tả thiết kế hệ thống của mình ở chương 3 và nguyên lý hoạt động ở chương 4. Chương 5 trình bày các kết quả thử nghiệm với bộ tăng tốc và phân tích. Chương 6 trình bày việc ứng dụng lõi tăng tốc cho bài toán phân loại cảm xúc khuôn mặt. Cuối cùng là kết luận của đề được trình bày trong chương 7.

## **1.2. Mục Tiêu Và Nhiệm Vụ Của Luận Văn:**

Các kiến thức và kết quả cần đạt được ở đề tài này là:

- Tìm hiểu giải thuật CNN, mô hình VGG16.
- Thiết kế kiến trúc phần cứng cho CNN.
- Hiện thực cấu trúc CNN trên FPGA.
- Thực hiện ứng dụng cấu trúc CNN trên FPGA để nhận dạng cảm xúc khuôn mặt.

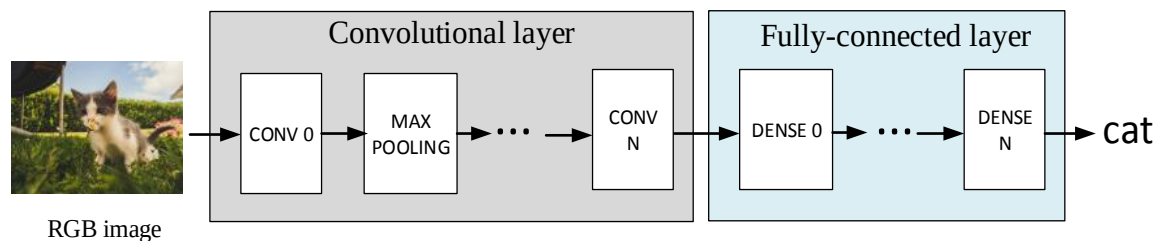
## CHƯƠNG 2: TỔNG QUAN

### 2.1 Mạng VGG16:

Mạng VGG16 được giới thiệu bởi Simonyan cùng các cộng sự [2] trong cuộc thi Image-Net LargeScale Vision Recognition Challenge (ILSVRC) diễn ra vào năm 2014. Mạng của họ đạt top-5 error là 7.32%, chỉ thua sau quán quân của cuộc thi năm đó là GoogleNet [14] với top-5 error là 6.67%. Mặc dù mạng VGG chứa số lượng tham số rất lớn nhưng cấu trúc lại đơn giản hơn rất nhiều so với GoogleNet. Trong hình 2.1, mạng VGG là một chuỗi các lớp tích chập và các lớp dense. Mỗi lớp tích chập dùng bộ lọc rất nhỏ có kích thước là 3x3. Stride và padding đều có giá trị cố định là 1 nhằm giữ lại nhiều thông tin giữa các kênh cũng như kích thước của ngõ ra sau khi thực hiện tích chập. Sau đó hàm activation là RELU (rectified linear unit) được dùng để tăng tính phi tuyến của mạng. Công thức của RELU như sau: (1)

$$\text{RELU}(x) = \max(0, x) \quad (1)$$

Max-pooling sau đó được dùng để giảm mẫu, một cửa sổ được trượt trên mỗi vùng ảnh có kích thước 2x2 với stride là 2. Điểm ảnh có giá trị cao nhất trong cửa sổ sẽ được chọn làm giá trị ngõ ra. Sau khi pooling, kích thước ngõ ra của feature maps được giảm đi một nửa so với input feature maps.



Hình 2.1 VGG like neural network.

Ta có thể thấy rằng mỗi lớp tích chập có cùng các thông số giống nhau như là cùng kích thước bộ lọc 3x3, stride và padding number và activation theo sau phép chập. Có 5 tham số thay đổi dọc theo các lớp CONV trong mạng VGG là input size, output size, số lượng input feature map, số lượng output feature map và có max pooling hay không, chi tiết trong bảng 2.1, trong đó input size và output size là kích thước của input và output feature maps, Nimap và Nomap là số lượng input và output

feature maps. Mỗi lớp CONV theo sau là một lớp max pooling được đánh dấu là 1 trong cột max pooling. Do đó, tôi đã đề xuất một bộ tăng tốc dựa trên lớp mạng có thể tái thiết lập cho mạng VGG, trong đó nó có thể thực hiện max pooling như một tùy chọn và điều chỉnh các tham số khác để thực hiện các lớp CONV khác nhau trong mạng VGG.

Bảng 2.1 Các thiết lập cho các lớp khác nhau của mạng VGG16.

| CONV layer | Input size | Output size | N <sub>imap</sub> | N <sub>omap</sub> | Max pooling |
|------------|------------|-------------|-------------------|-------------------|-------------|
| CONV1      | 224x224    | 224x224     | 3                 | 64                | 0           |
| CONV2      | 224x224    | 112x112     | 64                | 64                | 1           |
| CONV3      | 112x112    | 112x112     | 64                | 128               | 0           |
| CONV4      | 112x112    | 56x56       | 128               | 128               | 1           |
| CONV5      | 56x56      | 56x56       | 128               | 256               | 0           |
| CONV6      | 56x56      | 56x56       | 256               | 256               | 0           |
| CONV7      | 56x56      | 28x28       | 256               | 256               | 1           |
| CONV8      | 28x28      | 28x28       | 256               | 512               | 0           |
| CONV9      | 28x28      | 28x28       | 512               | 512               | 0           |
| CONV10     | 28x28      | 14x14       | 512               | 512               | 1           |
| CONV11     | 14x14      | 14x14       | 512               | 512               | 0           |
| CONV12     | 14x14      | 14x14       | 512               | 512               | 0           |
| CONV13     | 14x14      | 7x7         | 512               | 512               | 1           |

## 2.2 Các Kỹ Thuật Tối Ưu Vòng Lặp Trong Mạng CNN:

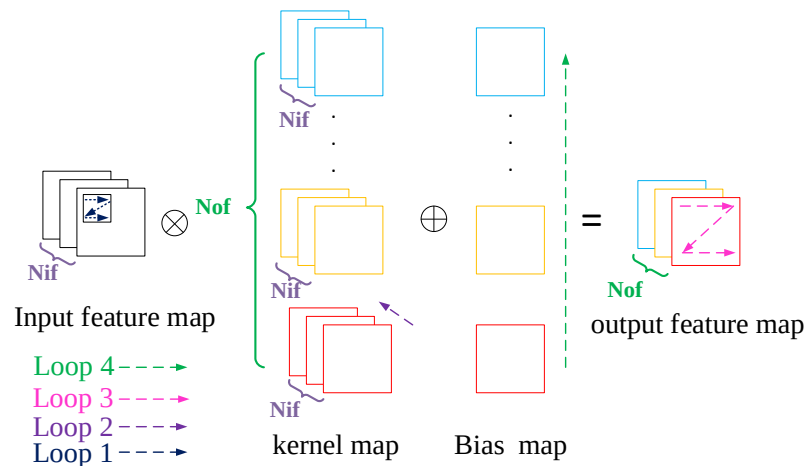
Tích chập là thành phần chính của lớp CONV, dữ liệu truyền vào là input feature map, sau đó thực hiện tích chập với kernel map nhằm tạo ra output feature map tạm thời. Những map tạm thời này sau đó được thêm vào map bias tương ứng nhằm tạo ra output feature map cuối cùng. Phép tính tích chập được trình bày ở công thức số (2), trong đó input feature map  $IN_j$  thứ  $j$  áp dụng tích chập cho  $G_{i,j}$  nhằm tạo ra output feature map  $OUT_i$  thứ  $i$ .

$$OUT_i = \sum_{j=1}^{N_{in}} IN_j \otimes G_{i,j} + B_i \quad (2)$$

Theo như Y. Ma cùng các đồng sự trong nghiên cứu [9] thì có bốn cấp độ vòng lặp trong lớp tích chập như hình 2.2. Trong đó,  $N_{in}$  và  $N_{of}$  tương ứng biểu diễn cho số lượng của input feature maps và output feature maps. Họ cũng đồng thời nghiên cứu một các kỹ lưỡng các kỹ thuật tối ưu vòng lặp đã được hiện thực trong các nghiên cứu trước đây [15 – 18]. Các kỹ thuật này là sự kết hợp của việc áp dụng kỹ thuật unrolling vào các lớp khác nhau, cụ thể như sau:

1. Type A: unroll loop-1, loop-2, loop-4

2. Type B: unroll loop-2, loop-4
3. Type C: unroll loop-1, loop-3
4. Type D: unroll loop-3, loop-4



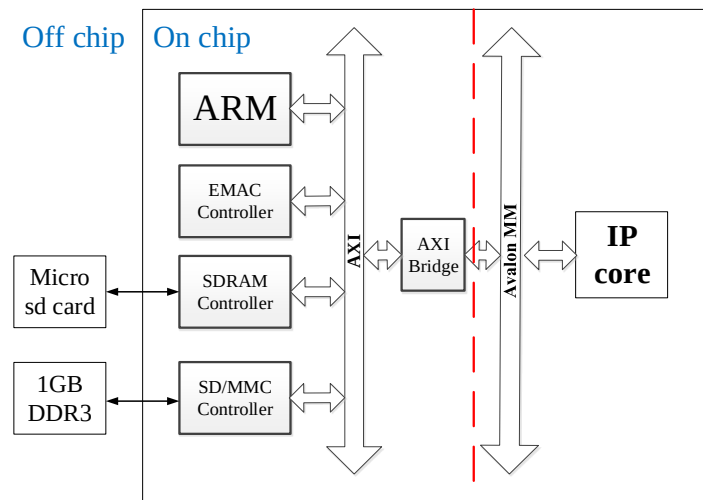
Hình 2.2 Bốn loại vòng lặp trong CNN.

Phân tích của họ cho thấy rằng loại D (unroll loop-3, loop-4) có ưu thế hơn so với các loại khác bởi vì nó có khả năng tái sử dụng lại dữ liệu cho cả input và weight. Thêm vào đó, loại D còn cho phép tính toán song song tốt hơn. Do đó, tôi quyết định sử dụng hướng tiếp cận loại D cho việc hiện thực bộ tăng tốc đồng thời cải tiến thêm nhằm tối đa hóa khả năng của toàn hệ thống.

### 2.3 DE10 Standard Board:

DE10 là một board FPGA SoC dựa trên dòng chip chi phí thấp của Altera là Cyclone V SoC (hình 2.3). Chip Cyclone V bao gồm 2 thành phần, được chia ra bởi đường kẻ màu đỏ. Phần bên trái là Hard Processing System (HPS) bao gồm lõi vi xử lý cứng ARM, bộ điều khiển EMAC, bộ điều khiển SDRAM,... trong khi phần bên phải là FPGA, được dùng để xây dựng các phần cứng tùy chỉnh. Hai thành phần này hoạt động trên những bus khác nhau, do đó một cầu AXI được dùng để giao tiếp giữa HPS logic và khối FPGA. Thêm vào đó, cầu AXI cung cấp một giao tiếp gọi là FPGA-to-HPS SDRAM, thông qua đó người dùng có thể truy cập trực tiếp vào bộ điều khiển SDRAM từ phía FPGA mà không cần thông qua hệ thống AXI của HPS. Giao tiếp này cho phép tối đa tới 4 Avalon MM read ports và 4 Avalon MM write ports, tổng độ rộng dữ liệu là 256 bit. Lợi ích này có thể tăng băng thông của bộ nhớ ngoài lên

tới 4.6GB/s. Tài nguyên sẵn có trên chip Cyclone V SoC FPGA được thể hiện trong bảng 2.2.



Hình 2.3 Cấu trúc chung của hệ thống Cyclone V SoC.

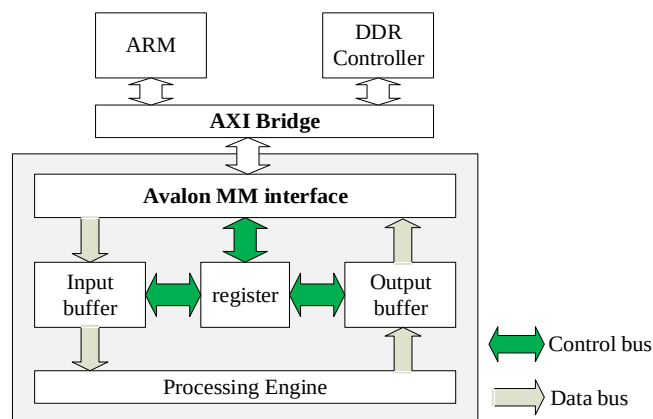
Bảng 2.2 Tài nguyên có sẵn trên chip Intel SoC FPGA Cyclone V.

| Resource            | Available |
|---------------------|-----------|
| ALMs                | 41509     |
| DSP                 | 112       |
| On chip memory (Kb) | 5570      |

## CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

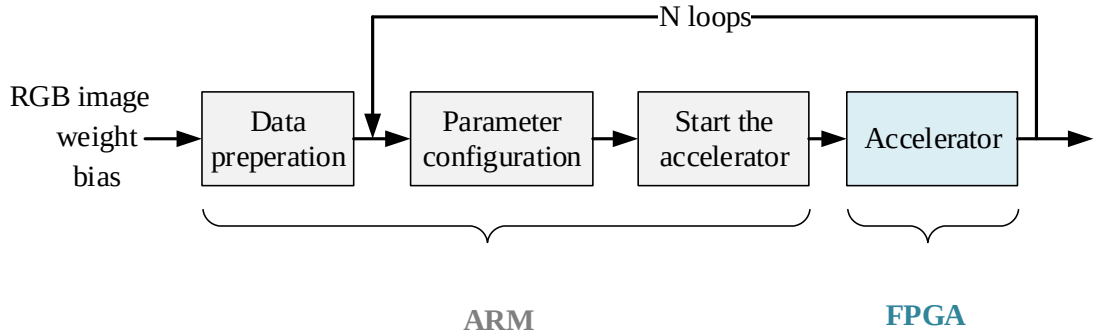
### 3.1 Kiến Trúc Chung:

Hình 3.1 thể hiện kiến trúc chung của bộ tăng tốc cho mạng VGG bao gồm: Input Buffer (IB), Output Buffer (OB), Processing Engine (PE), thanh ghi và giao tiếp Avalon MM. Lỗi ARM kết hợp với bộ tăng tốc tạo thành một hệ thống tính toán bất đồng bộ, trong đó tôi dùng 1G DDR3 RAM làm bộ nhớ chia sẻ. Nhiệm vụ của lỗi ARM là điều khiển trạng thái hoạt động của bộ tăng tốc bằng việc đọc và ghi dữ liệu và các thanh ghi điều khiển trong khối Register. Lỗi ARM cũng đồng thời đảm nhiệm việc tiền xử lý các dữ liệu phục vụ cho giai đoạn đầu của quá trình dự đoán như là ghi ảnh, kernel, bias map vào trong bộ nhớ RAM. Các hoạt động còn lại bao gồm zero padding, tính tích chập, RELU và max-pooling được thực hiện bởi bộ tăng tốc.



Hình 3.1 Kiến trúc chung của bộ tăng tốc VGG.

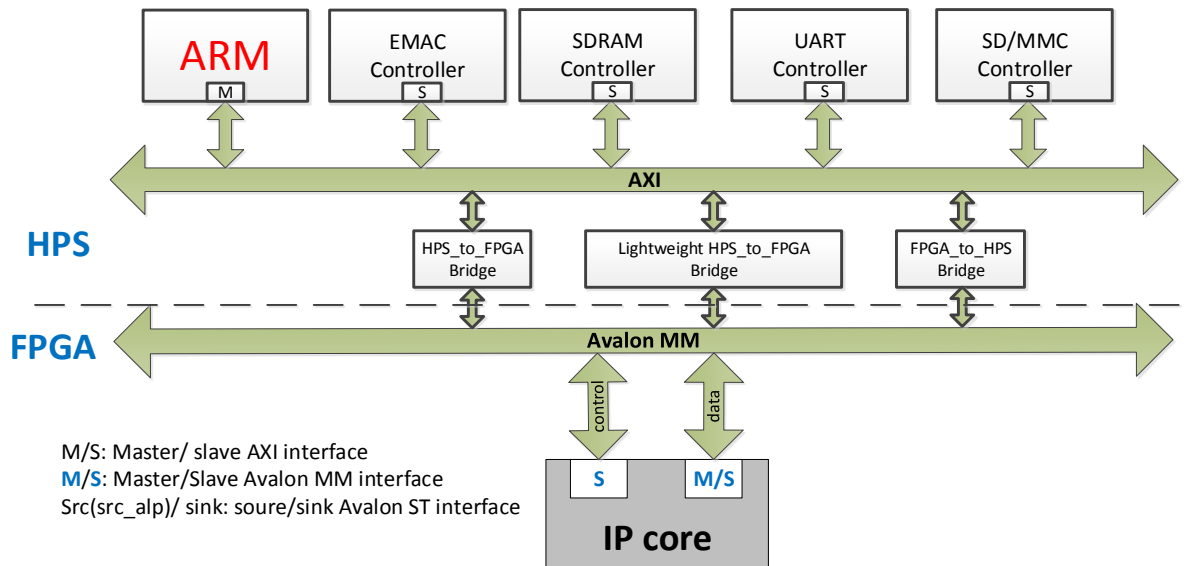
Quy trình cho việc dự đoán được mô tả trong hình 3.2, trong đó N là số lượng lớp CONV. Ta cũng có thể thấy trong hình 3.2 là bộ tăng tốc chỉ có thể hiện thực một lớp tại một thời điểm, do đó chương trình chạy trên ARM cần phải tái thiết lập lại các tham số cho các lớp phía sau sau khi bộ tăng tốc kết thúc tính toán cho lớp hiện tại. Giả sử rằng hoạt động thiết lập các tham số cần vài chu kỳ máy để chuyển dữ liệu điều khiển từ ARM xuống thanh ghi. Kết quả là thời gian bị chiếm bởi việc xử lý trên phần mềm là không đáng kể.



Hình 3.2 Trình tự hoạt động của vi điều khiển và bộ tăng tốc.

### 3.2 Sơ Đồ Thiết Kế Lõi IP CNN Trong Hệ Thống Soc Cyclone V:

Hình 3.3 mô tả chi tiết kiến trúc toàn hệ thống SoC. Trong đó hệ thống được chia thành hai thành phần chính là HPS và FPGA, hai thành phần này giao tiếp với nhau qua 3 cầu AXI: HPS\_to\_FPGA; Lightweight HPS\_to\_FPGA và FPGA\_to\_HPS.

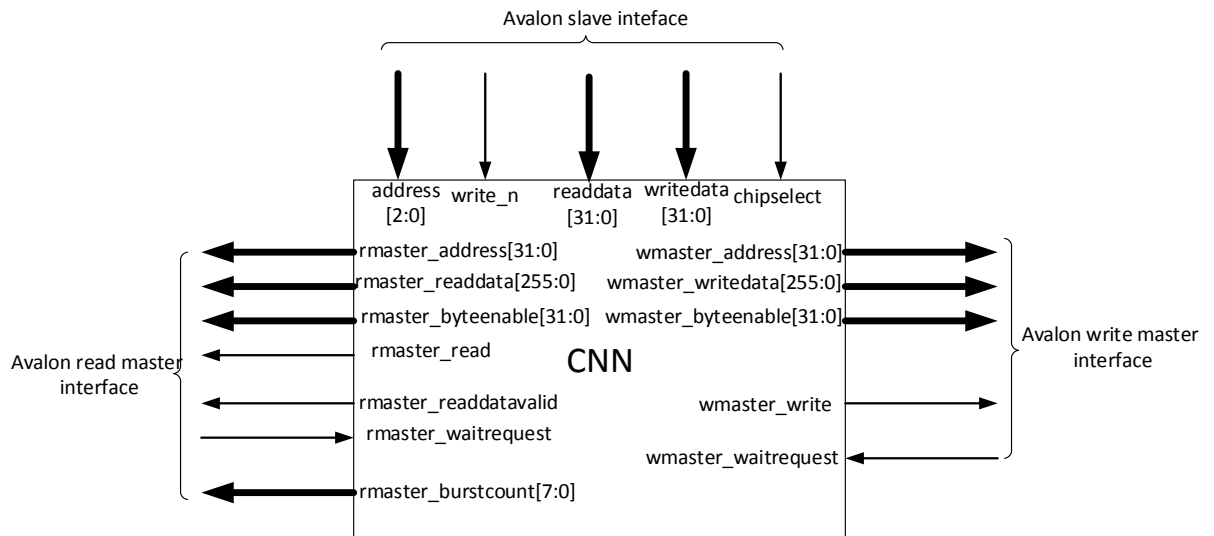


Hình 3.3 Chi tiết kiến trúc hệ thống.

Hình 3.4 mô tả khối Top level của lõi CNN, các luồng dữ liệu vào và ra đều dùng giao tiếp Avalon. Trong đó:

- Giao tiếp Avalon slave : nhận các tín hiệu điều khiển thanh ghi từ vi xử lý
- Giao tiếp Avalon read/write master (slave): truyền/ nhận dữ liệu (input feature map, weight, bias). Có 2 cách ghi/đọc dữ liệu từ CNN core:

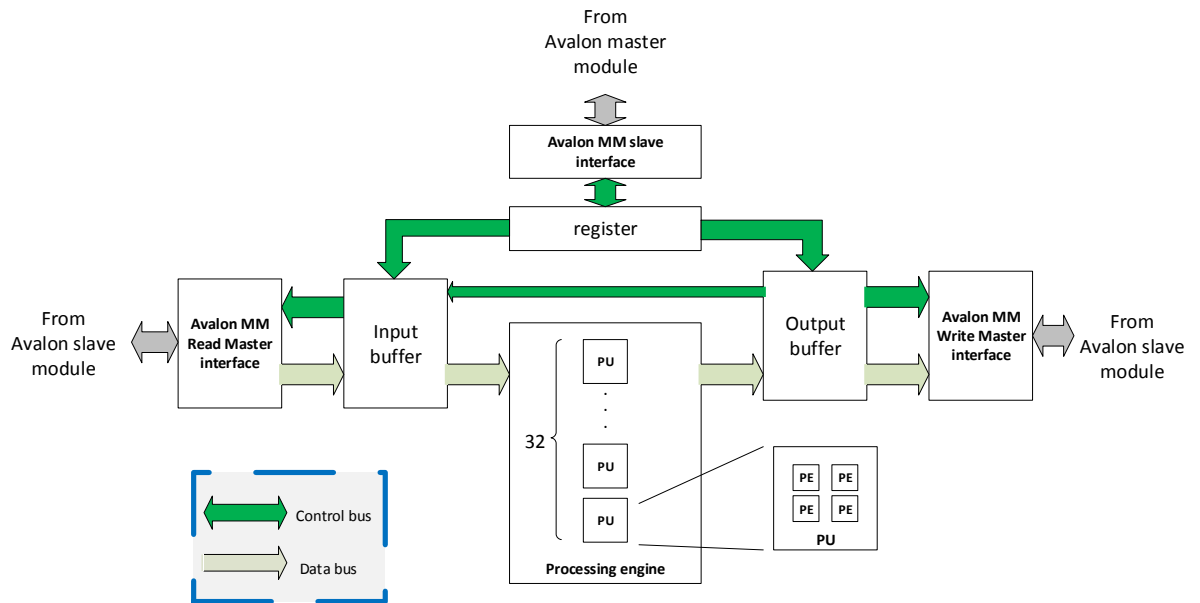
- Dùng vi xử lý ARM để truyền/ nhận dữ liệu tốc độ cao thông qua cầu HPS\_to\_FPGA bridge.
- Lõi CNN truy xuất trực tiếp vào bộ nhớ DDR3 RAM bên phía HPS qua cầu Sdram FPGA\_to\_HPS



Hình 3.4 Sơ đồ Top-level lõi CNN.

Bên trong của top module là các khối con khác (hình 3.5), mỗi khối con đảm nhận những chức năng khác nhau trong quá trình xử lý các lớp tích chập. Sau đây là sơ lược chức năng của từng module này:

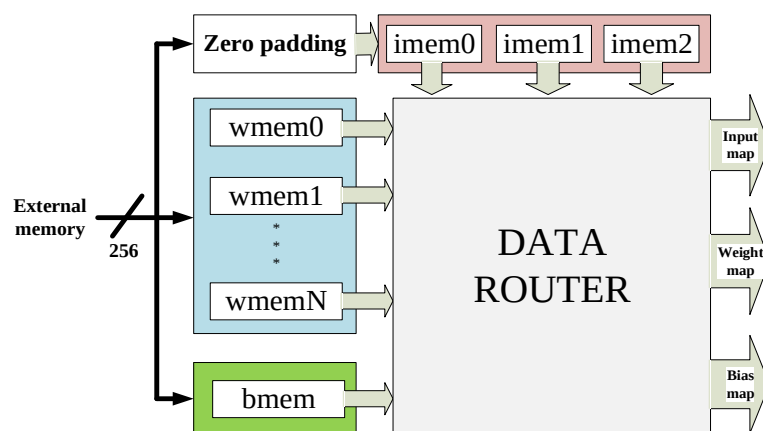
- Giao tiếp Avalon (master/slave): điều khiển các tín hiệu giao tiếp Avalon.
- Input buffer: bộ đệm chứa dữ liệu input feature map, weight, bias; đồng thời điều khiển truyền dữ liệu vào bộ Processing Engine.
- Processing engine: bộ tính toán, bao gồm 32 bộ PU, mỗi bộ PU gồm 4 bộ PE, mỗi PE là một bộ accumulator (nhân và cộng dồn).
- Output buffer: bộ đệm dữ liệu ngõ ra từ Processing Engine.



Hình 3.5 Sơ đồ khối các module của lõi CNN.

### 3.3 Input Buffer (IB):

IB lưu trữ dữ liệu hình ảnh, bias và trọng số đọc từ bộ nhớ ngoại (hình 3.6). IB cũng thực hiện việc zero padding cho input feature map, công việc này thường được thực hiện bằng phần mềm. Imem, wmem, bmem đóng vai trò làm on chip buffer nhằm lưu input feature map, trọng số và bias. IB được tạo từ bộ nhớ RAM 2 port (2-PORT RAM) với độ rộng dữ liệu đọc và ghi khác nhau. Tôi thiết lập cho port ghi dữ liệu tới 256 bit nhằm tương thích với port đọc Avalon MM đã thảo luận trong phần 2.3. Khối Data Router nhận dữ liệu từ buffer và sắp xếp lại trước khi nạp vào khối xử lý trung tâm Processing Engine.



Hình 3.6 Thiết kế Input Buffer.

Có  $N$  wmems lưu trữ  $N$  các kernel map khác nhau trong lớp CONV. Mỗi kernel thực hiện tính tích chập với input feature map và sẽ trả lại kết quả là output feature map tương ứng. Do đó, bộ tăng tốc cần  $Nof/N$  vòng nhằm tạo ra kết quả hoàn chỉnh. Kích thước của từng wmem được tính trong công thức (3), trong đó số PREC là số byte biểu diễn trọng số và dữ liệu,  $Nif\_max$  là giá trị tối đa trong các giá trị  $Nif$  của tất cả các lớp CONV trong mạng.

$$S_{wmem} = PREC * 9 * Nif\_max \quad (3)$$

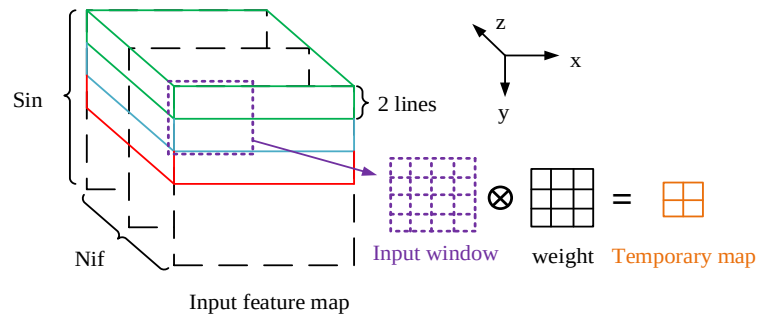
Các số bias của các lớp CONV chiếm một tỉ lệ rất nhỏ trong tổng số các tham số, do đó các giá trị bias được đọc một lần duy nhất khi bắt đầu, lưu trong bmem, tái sử dụng và được hủy sau khi kết thúc quá trình tăng tốc. Cũng như với wmem, kích thước của bmem được mô tả trong công thức (4), trong đó  $Nof\_max$  là giá trị lớn nhất giữa các  $Nof$  của tất cả các lớp CONV trong mạng.

$$S_{bmem} = PREC * Nof\_max \quad (4)$$

Bởi vì giới hạn của bộ nhớ trên chip, các input feature map được chia ra thành các hộp màu như mô tả trong hình 3.7. Mỗi hộp chứa 2 dòng của mỗi input feature map và được lưu trong imem. Do đó, không gian lưu trữ cần thiết của imem được mô tả trong công thức (5), trong đó  $Sin$  là kích thước của các input feature map có dạng vuông.

$$S_{imem} = PREC * 2 * Sin * Nif\_max \quad (5)$$

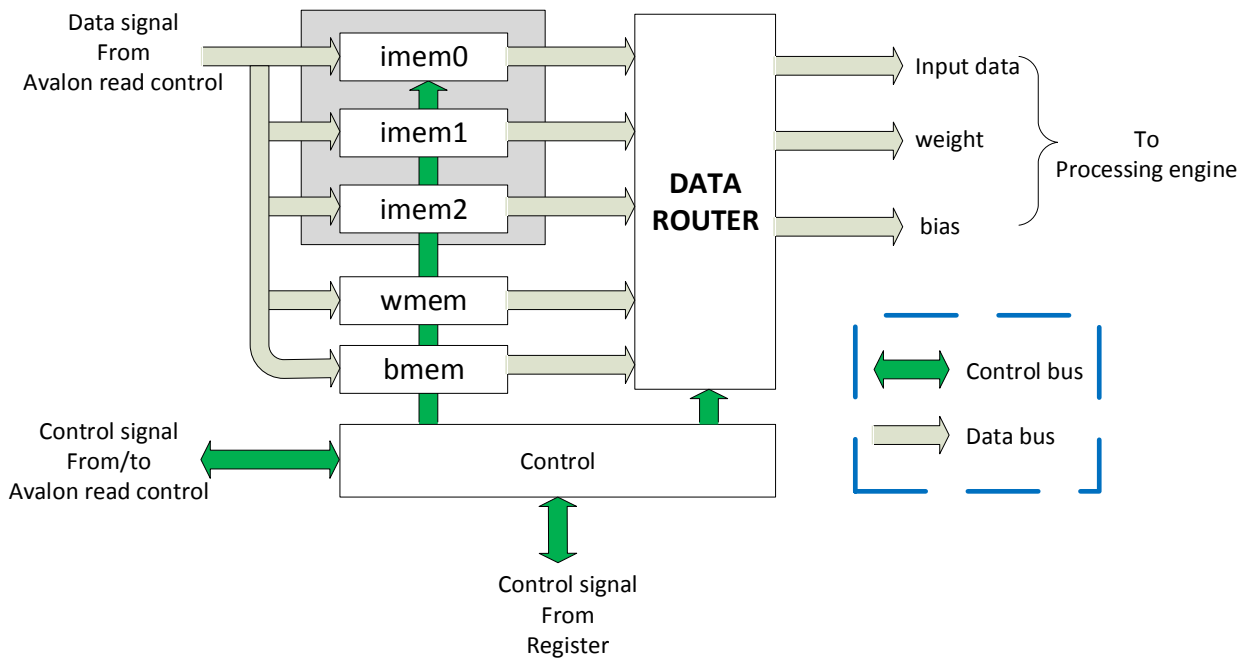
Sự kết hợp của 2 imems tạo thành cửa sổ ngõ vào (hình 3.7) có kích thước  $4 \times 4$ . Cửa sổ này sẽ di chuyển dọc theo tất cả input feature map dọc theo trục  $z$  và thực hiện tính tích chập với trọng số nhằm tạo ra các map tạm  $Nif$ . Những map này được cộng lại cùng nhau và với map bias để tạo ra output feature map kích thước  $2 \times 2$ . Sau đó, cửa sổ di chuyển dọc theo trục  $x$  với bước nhảy là 2 và lặp lại quá trình nhằm tạo ra những output feature map kích thước  $2 \times 2$  khác.



Hình 3.7 Quá trình thực hiện tích chập giữa trọng số và box (input window) từ các kênh màu.

Nhằm tăng cường throughput của hệ thống, tôi dùng imem thứ ba cho dữ liệu mới trong khi 2 imem còn lại đang hoạt động. Do đó, dữ liệu ngõ vào luôn sẵn sàng cho việc nạp vào khối Processing Engine. Khi cửa sổ (hình 3.7) chạm tới điểm cuối cùng bên phải của input feature map, tất cả dữ liệu trong imem đầu tiên bị xoá đi và imem được nạp vào dữ liệu của box tiếp theo. Imem thứ hai kết hợp với imem thứ ba nhằm tạo ra thêm 2 hàng output feature map. Quá trình này lặp lại cho đến khi tất cả các box được đọc từ bộ nhớ ngoài.

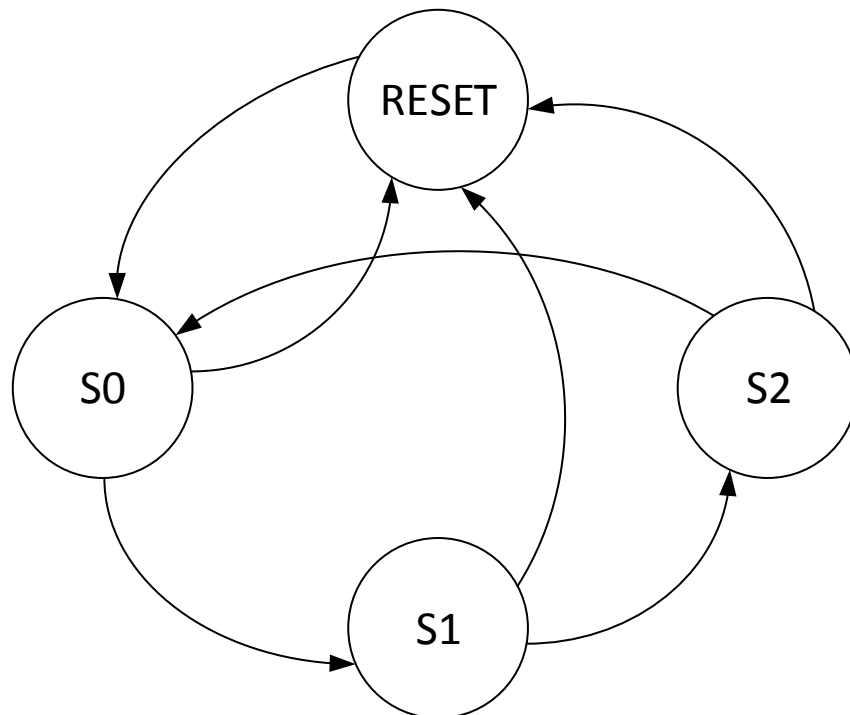
Sơ đồ khối của Input Buffer được mô tả chi tiết như hình 3.8:



Hình 3.8 Sơ đồ khối Input Buffer.

**imem** : bộ nhớ chứa input feature map, sử dụng 2-PORT RAM của Altera, trong đó port write có bộ rộng data bus là 256, port read data có độ rộng bus là 64. Trong

một lần khối Processing Engine tính toán thì cần đọc cùng lúc 2 imem, imem thứ 3 là vùng đệm của lần xử lý kế tiếp để tăng hiệu suất tính toán. Quá trình đọc các imem theo máy trạng thái (hình 3.9), trong đó S0: đọc 2 imem0 và imem1, S1: đọc imem1 và imem2, S2: đọc imem2 và imem0. Mỗi lần chuyển trạng thái giữa các trạng thái S\_x thì imem thứ nhất (imem0) bị clear (ví dụ trạng thái S0 chuyển sang trạng thái S1 thì imem0 bị clear) và sẵn sàng ghi vào dữ liệu kế tiếp. Sau khi xử lý xong tất cả các input feature map thì các trạng thái S\_x trở về trạng thái reset.



Hình 3.9 Máy trạng thái Input Buffer.

**wmem:** bộ nhớ chứa các trọng số weights, sử dụng 2-PORT RAM của Altera, trong đó port write có bộ rộng data bus là 256, port read data có độ rộng bus là 144 (16 bit/pixel x 9 weights). Trong đó có 32 bộ đệm con để lưu 32 kernel khác nhau của weights.

**bmem:** bộ nhớ chứa các giá trị bias, sử dụng 2-PORT RAM của Altera, trong đó port write có bộ rộng data bus là 256, port read data có độ rộng bus là 256.

**Data router:** có chức năng sắp xếp các data đọc từ bộ đệm và di chuyển đến Processing Engine.

Chi tiết kích thước của input buffer trong thiết kế như sau:

$$S_{wmem} = PREC * 9 * Nif\_max = 16 * 9 * 512 = 73728 \text{ bit (9 KB)}$$

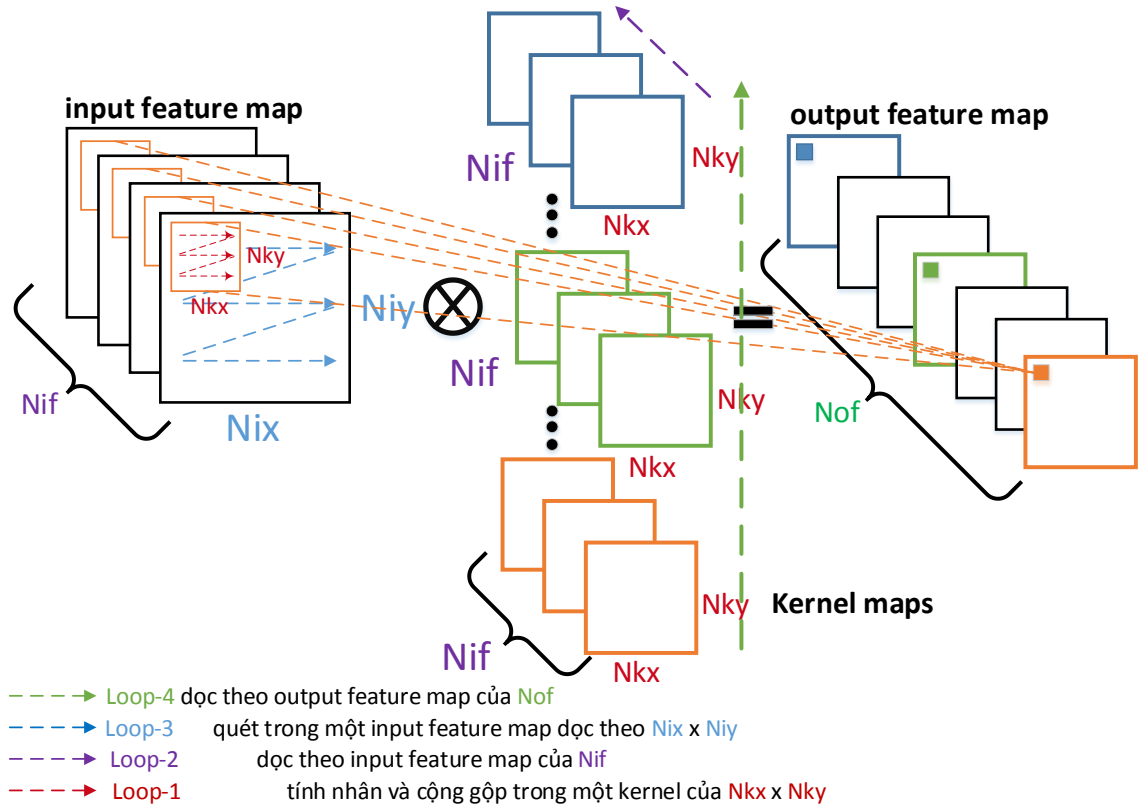
$$S_{bmem} = PREC * Nof\_max = 16 * 512 = 8192 \text{ bit (1 KB)}$$

$$S_{imem} = PREC * 2 * Sin * Nif\_max = 16 * 2 * 224 * 512 = 3670016 \text{ bit (448 KB)}$$

$$\text{Buffer size} = S_{wmem} + S_{bmem} + S_{imem} = 458 \text{ KB}$$

### 3.4 Processing Engine (PE):

Hình 3.10 mô tả cách các vòng lặp hoạt động trong lớp tích chập. Các vòng lặp này được chia thành 4 loại dựa theo cấp độ vòng lặp từ cao xuống thấp.



Hình 3.10 Mô tả và phân loại các vòng lặp trong lớp chập.

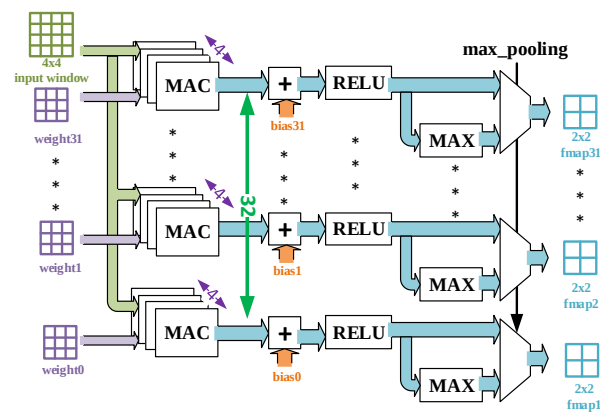
Processing engine (PEn) là một trong các thành phần quan trọng nhất trong bộ tăng tốc. Chức năng của PE là thực hiện tính tích chập giữa input feature map và trọng số sau đó trả về output feature map. Trong hình 3.11, tôi thiết kế PE chuyên dụng cho các lớp tích chập của mạng VGG16. Tuy nhiên, cấu trúc này có thể áp dụng cho các mạng khác dựa trên mạng VGG16 bằng việc lựa chọn số lượng của các MACs (Multiplier and Accumulator unit). Bởi vì thiết kế của tôi dựa trên kỹ thuật tối ưu vòng lặp loại D như đã đề cập ở phần trên, tôi định ra 2 biến là Cxy và Cof biểu thị số lượng tính toán song song cho vòng lặp 3 và vòng lặp 4. Để tương thích với Input

B, Cxy được cố định là 4 và Cof được tính toán tùy thuộc vào Nof nhỏ nhất và tài nguyên có sẵn của FPGA. Tổng số MAC được tính bằng công thức (6).

$$\text{num\_macs} = C_{xy} \times C_{of} = 4 \times C_{of} \quad (6)$$

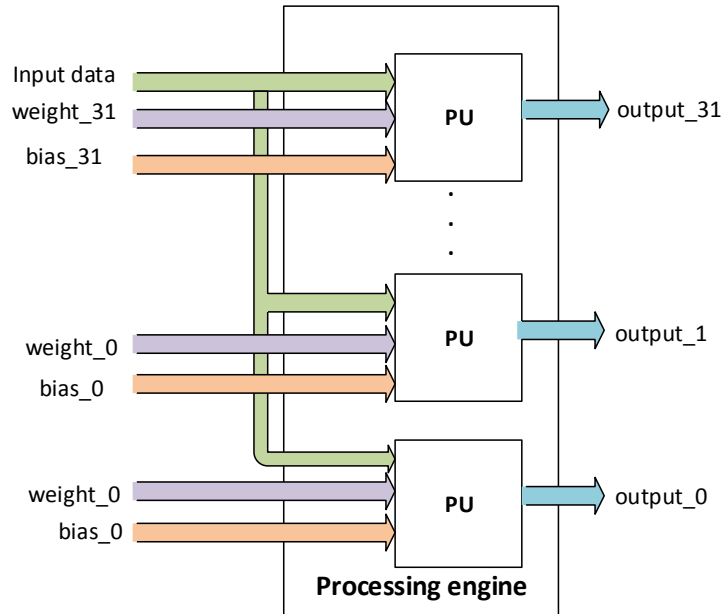
Khối này được thiết kế dựa trên kiểu thiết kế pipeline. Mất tất cả  $(9 \times N_{if} + 1)$  chu kỳ máy để thực hiện tích chập và thêm 2 chu kỳ nữa cho các hoạt động khác như RELU và maxpooling để tạo các pixel đầu ra num\_macs. Do đó, tổng thời gian để xử lý một lớp CONV được tính bằng công thức (7), trong đó DDR\_time là thời gian truy cập bộ nhớ ngoài của bộ tăng tốc, biến max được đặt thành 1 khi các lớp CONV thực maxpooling và 0 trong trường hợp ngược lại.

$$\text{layer\_time} = (9 \times N_{if} + 2 + \max) \times (\text{Sin} \times \text{Sin} / \text{num\_macs}) \times 2^{(-2 \times \max)} + \text{DDR\_time (cycles)} \quad (7)$$



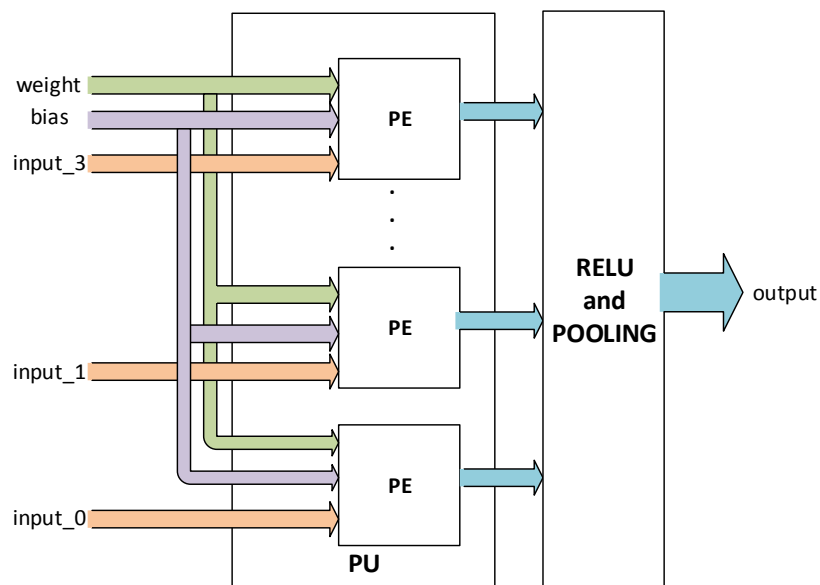
Hình 3.11 Processing Engine cho VGG16.

Dữ liệu được truyền từ input buffer vào PU bao gồm input data, trọng số và bias. Từ hình 3.12 có tất cả 32 Processing Unit (PU) trong Processing Engine, 32 PU này tính toán song song, mỗi bộ PU có chức năng tính tích chập từ input feature map với một kernel và cho kết quả là output feature map của kernel đó. Việc song song các PU kỹ thuật unroll loop 4, sơ đồ chi tiết:



Hình 3.12 Sơ đồ khối dữ liệu vào Process Engine và các Processing Unit.

Mỗi PU gồm 4 PE (hình 3.13). Mỗi PE bao gồm một bộ nhân có dấu 16-bit và 2 bộ cộng 32-bit. Việc thực hiện tính toán song song của các PE đã áp dụng kỹ thuật unroll loop 3. Kết quả từ các PE sau đó sẽ được tính Relu và Max Pooling.



Hình 3.13 Process Engine với Relu và Max Pooling.

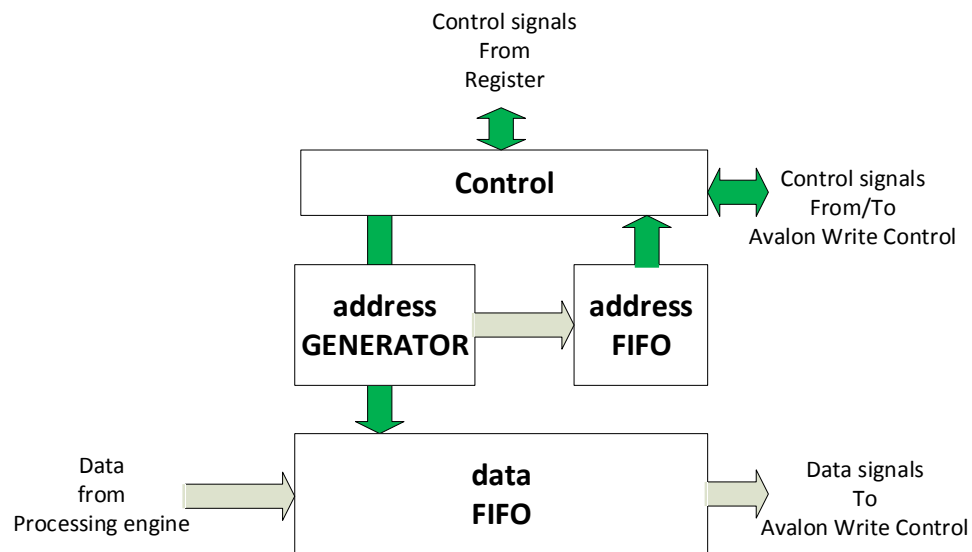
Trên board DE-10 có tất cả 112 DSP, tuy nhiên chúng ta cần tổng cộng 128 DSP cho 128 PE. 16 DSP còn thiếu sẽ được Quartus tổng hợp từ tài nguyên của FPGA (soft DSP).

### 3.5 Output Buffer (OB):

Tôi quyết định chuyển các output feature pixel ngay lập tức sang bộ nhớ ngoài mà không cần chờ toàn bộ map hoàn chỉnh. Do đó, không cần on-chip buffer lớn để lưu trữ kết quả đầu ra. Output buffer còn có chức năng của FIFO, là một bộ đồng bộ hóa dữ liệu do sự khác biệt về tốc độ truyền dữ liệu giữa PEn và giao tiếp Avalon MM.

Sơ đồ khối của Output Buffer được trình bày chi tiết trong hình 3.14, với các khối và chức năng như sau:

- Address generator: Tạo địa chỉ tương ứng cho data từ Processing engine, địa chỉ này là địa chỉ ghi vào vùng nhớ ngoài (SDR hoặc DDR RAM).
- Address FIFO và data FIFO: bộ FIFO chứa địa chỉ và data tương ứng.
- Control: bộ điều khiển hoạt động cho Output buffer.



Hình 3.14 Sơ đồ khối Output Buffer.

### 3.6 Register:

Khối này là nơi lưu trữ tất cả các thông số hoạt động của bộ tăng tốc. Những dữ liệu này được truy cập bởi cả ARM và bộ tăng tốc thông qua 5 thanh ghi 32 bit có tên LAYER\_STATUS, START, DONE, RADDR, WADDR.

- Thanh ghi LAYER\_STATUS chứa các tính năng của các lớp CONV bao gồm kích thước của input feature map (Sin), số lượng input feature map (Nif), số lượng output feature map (Nof) và tùy chọn có max-pooling hay không.

- Thanh ghi START cho phép người dùng khởi động bộ tăng tốc bằng cách ghi giá trị 1 vào thanh ghi này.
- Thanh ghi DONE được đặt thành 1 bởi bộ tăng tốc cho biết việc tính toán đã hoàn tất.
- Thanh ghi RADDR và WADDR mô tả địa chỉ bắt đầu của bộ nhớ ngoài để lưu input và output feature map.

Chi tiết các thanh ghi được trình bày trong bảng 3.1:

Bảng 3.1 Chi tiết các thanh ghi điều khiển.

| Địa chỉ | Tên thanh ghi | Loại | Mô tả                                                                                                                                                                                                                                                                                                                                     |
|---------|---------------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0       | START         | WO   | Bit 0 set lên 1 cho phép lõi IP bắt đầu hoạt động                                                                                                                                                                                                                                                                                         |
| 1       | MEM STATUS    | RO   | <ul style="list-style-type: none"> <li>- Bit 0 lên '1' khi bộ đệm input feature map thứ 1 full</li> <li>- Bit 1 lên '1' khi bộ đệm input feature map thứ 2 full</li> <li>- Bit 2 lên '1' khi bộ đệm input feature map thứ 3 full</li> <li>- Bit 3 lên '1' khi bộ đệm weight full</li> <li>- Bit 4 lên '1' khi bộ đệm bias full</li> </ul> |
| 2       | LAYER STATUS  | R/W  | Chứa các đặc tính của layer đang thực thi <ul style="list-style-type: none"> <li>- Bit 31:</li> <li>- Bit 30: '1' → max pooling</li> <li>- Bit 29-20: kích thước input feature map</li> <li>- Bit 19-10: số lượng kernel của weight</li> <li>- Bit 9-0: số lượng channel của weight</li> </ul>                                            |
| 3       | DONE          | RO   | '1' → Hoàn thành thực thi một lớp conv                                                                                                                                                                                                                                                                                                    |

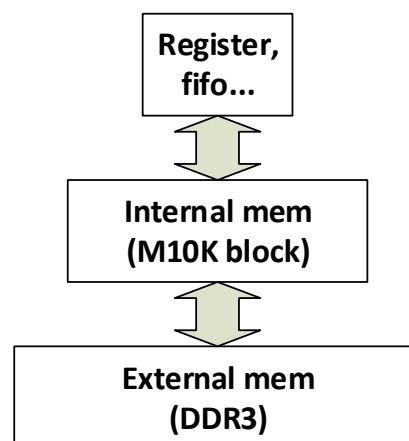
|   |       |     |                                                                 |
|---|-------|-----|-----------------------------------------------------------------|
| 4 | WBASE | R/W | Thanh ghi chứa địa chỉ base của output feature map trên DDR Ram |
|---|-------|-----|-----------------------------------------------------------------|

### 3.7 Giao Tiếp Avalon MM:

Giao tiếp Avalon MM được dùng để giao tiếp với các thành phần khác nhau trong hệ thống Intel-FPGA. Giao tiếp này bao gồm một module master và một module slave. Module master có hiệu suất cao nhằm chuyển các input feature map, trọng số và output feature map giữa bộ nhớ ngoài và bộ tăng tốc trong khi module slave được sử dụng để tương tác với bộ vi xử lý.

### 3.8 Memory:

Có 3 cấp độ bộ nhớ được dùng trong thiết kế (Hình 3.15) là: Bộ nhớ ngoài (external memory) được dùng cho cả FPGA và HPS, bộ nhớ nội (bên trong phần FPGA) là các khối M10K và cấp cuối cùng là các thanh ghi và FIFO.

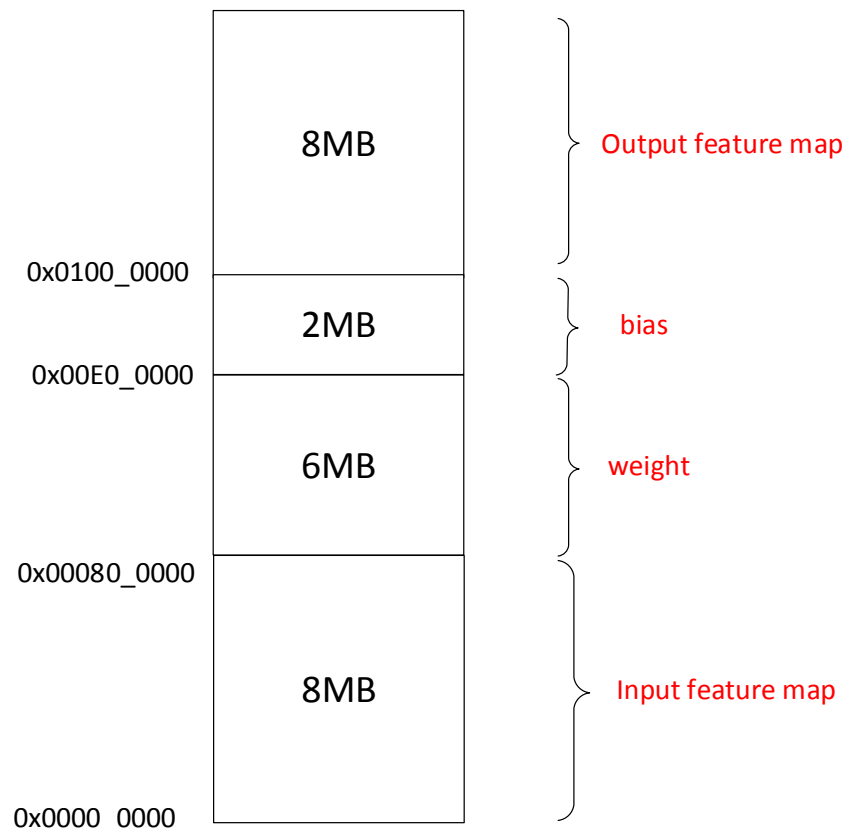


Hình 3.15 Phân tầng vùng nhớ.

#### 3.8.1 External Mem:

External mem là bộ ngoại vi có kích thước lớn, trong trường hợp này là DDR3 (HPS-side) của kit DE10. Bộ nhớ này được dùng làm Shared memory-vùng nhớ dùng chung cho cả lõi phần cứng CNN và vi xử lý ARM. Vùng nhớ này có kích thước là 64MB, có địa chỉ base là 0x300\_0000, chi tiết như hình 3.16.

Tổ chức vùng nhớ Shared memory:



Hình 3.16 Tổ chức vùng nhớ Shared memory.

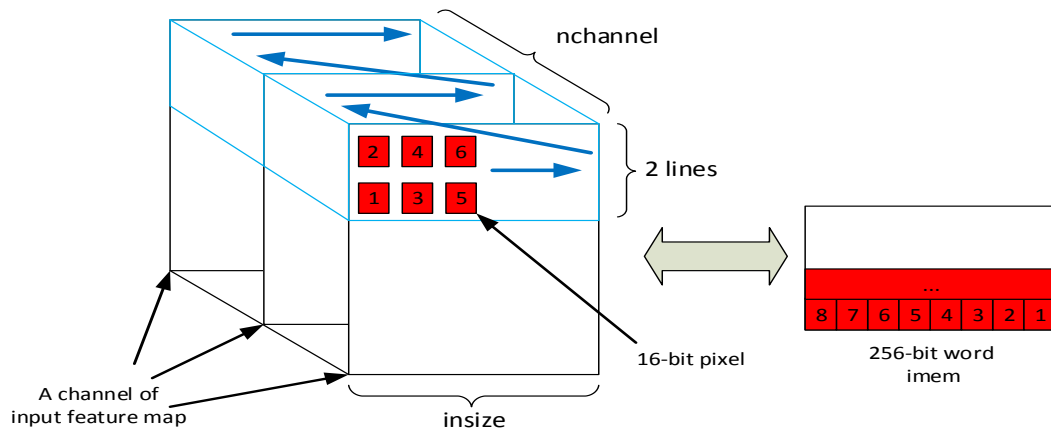
### 3.8.2 On-Chip Memory:

On-chip memory là vùng nhớ đệm bên trong lõi phần cứng CNN. Vùng nhớ này để lưu dữ liệu ngõ vào (input feature map, bias, weight) và ngõ ra (output feature map). Kích thước khoảng vài trăm kb.

### 3.8.3 Input Feature Map Memory:

Là vùng nhớ on-chip để lưu các thông tin input feature map

Vùng nhớ này gồm 3 bộ đệm riêng biệt, mỗi bộ đệm là 2-PORT RAM (IP core của altera) lưu trữ 2 lines x nchannel (pixels) của input feature map. Vì bus data của port Avalon read có độ rộng là 256 nên mỗi write-word của imem có bộ rộng là 256bit, tương đương với 8 pixels. Hình 3.17 mô tả cách sắp xếp dữ liệu input feature map vào imem.

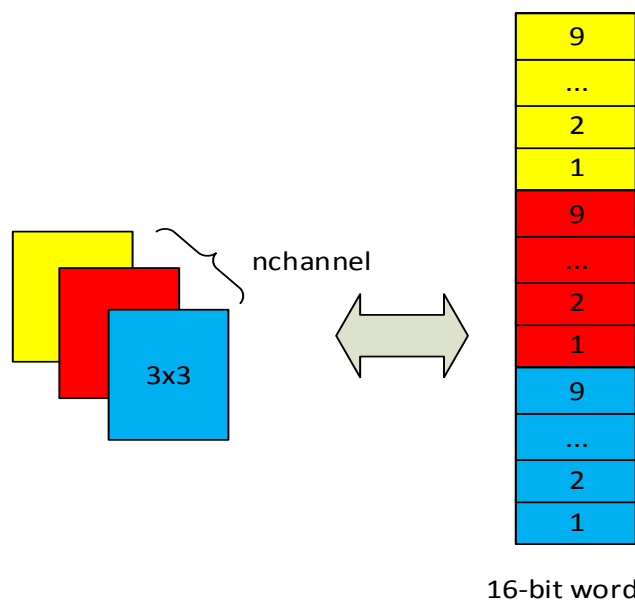


Hình 3.17 Cách sắp xếp dữ liệu input feature map vào imem.

### 3.8.4 Weights Memory:

Weights memory (wmem) là vùng nhớ đệm cho có trọng số. Có tổng cộng 32 bộ đệm, mỗi bộ đệm là 2-PORT RAM chứa một kernel của trọng số trong 1 layer (1 kernel gồm  $nchannel \times 3 \times 3$  pixels). Bên dưới là cấu trúc vùng nhớ của của một bộ đệm lưu các giá trị trọng số của kernel tương ứng.

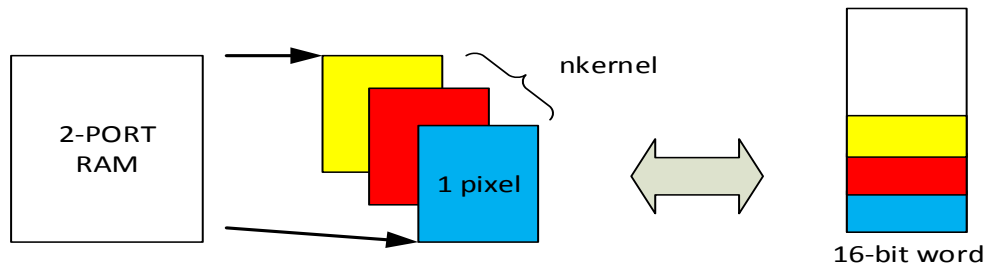
Hình 3.18 mô tả cách sắp xếp dữ liệu trọng số vào bộ nhớ wmem:



Hình 3.18 Cách sắp xếp dữ liệu trọng số vào bộ nhớ wmem.

### 3.8.5 Bias Memory:

Bias memory là vùng nhớ để chứa tất cả các bias trong một layer. Hình 3.19 mô tả cách sắp xếp dữ liệu bias vào bộ nhớ.

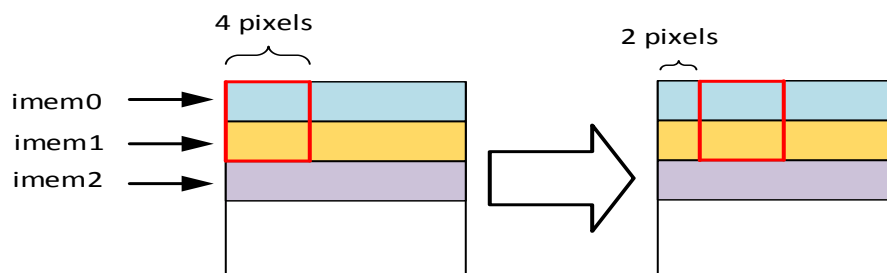


Hình 3.19 Tổ chức vùng nhớ chứa bias.

## CHƯƠNG 4: NGUYÊN LÝ HOẠT ĐỘNG

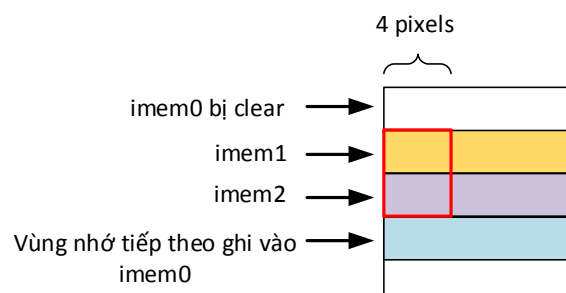
### 4.1 Nguyên Lý Hoạt Động Của Lỗi Tăng Tốc:

Vùng nhớ input feature map gồm 3 bộ đệm là imem0, imem1, imem2. Mỗi lần đọc dữ liệu cho bộ Processing engine cần 2 imem, imem thứ 3 là bộ đệm tạm thời để tăng độ throughput. Khung màu đỏ là phần dữ liệu được đọc ra từ 2 imem. Khung màu đỏ là vùng pixel có kích thước là 4x4, mỗi lần trượt là 2 pixel đến hết bộ nhớ của imem. Hình 4.1 và 4.2 trình bày chi tiết kỹ thuật giúp tăng throughput toàn hệ thống:



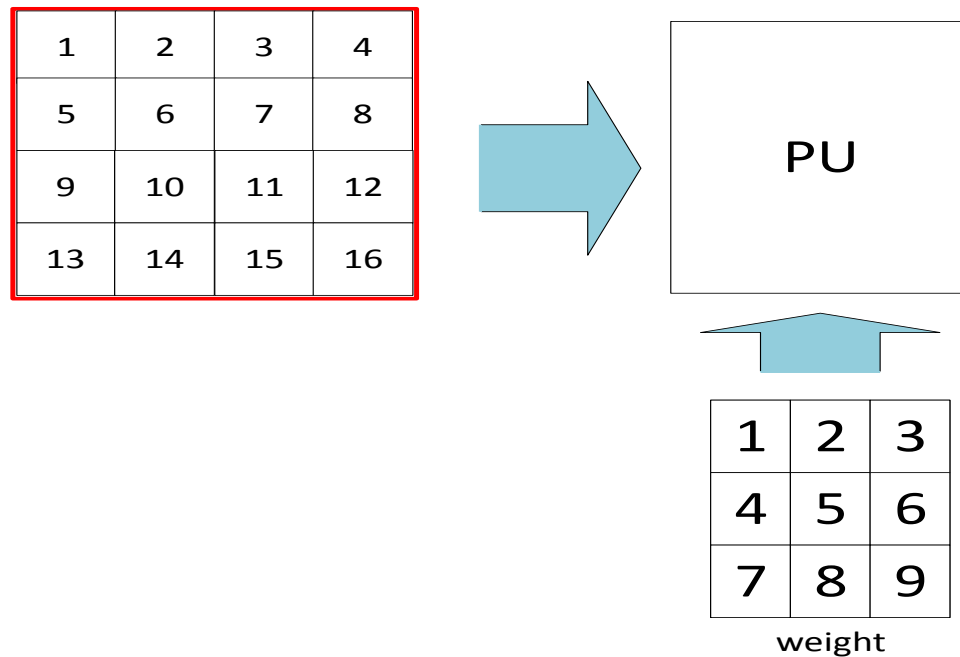
Hình 4.1 Cửa sổ trượt đồng thời trên 2 imem 1 lúc.

Sau khi đọc hết dữ liệu của 2 imem, imem thứ 3 được sử dụng, imem thứ 1 bị clear và chuẩn bị ghi vào dữ liệu mới.



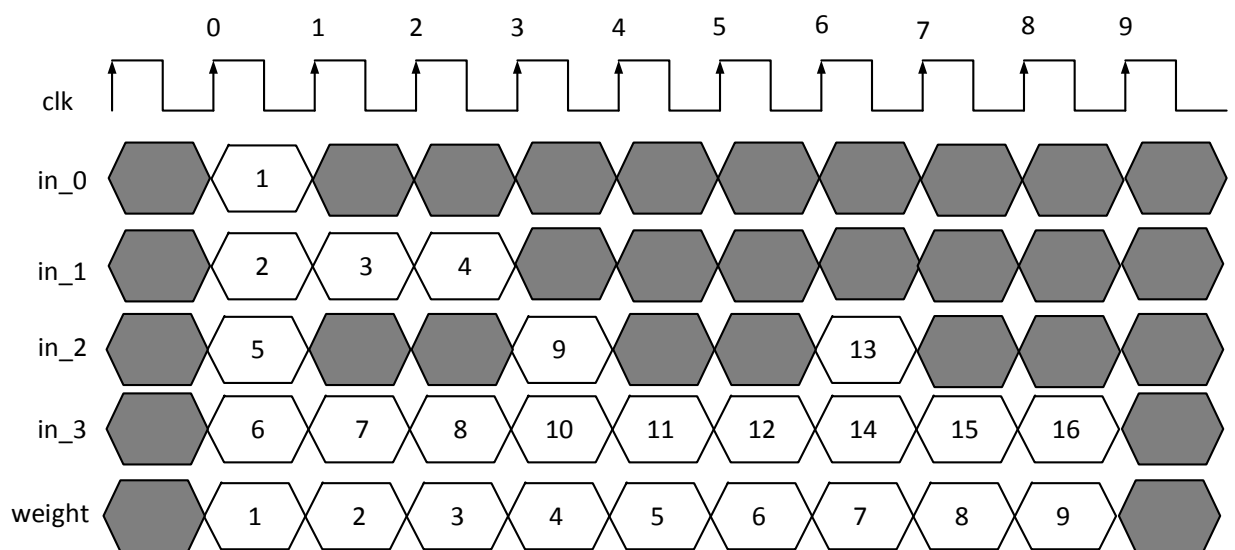
Hình 4.2 Lần trượt thứ 2 sau khi đã đi hết 2 imem 0 và 1.

Vùng nhớ được đọc ra từ imem (khung màu đỏ gồm 16 pixels) và weight được đưa đến bộ data router để feed vào bộ PU trong Processing engine. Hình 4.3 mô tả cách dữ liệu từ imem được đưa vào PU.



Hình 4.3 Dữ liệu từ imem được đưa vào PU.

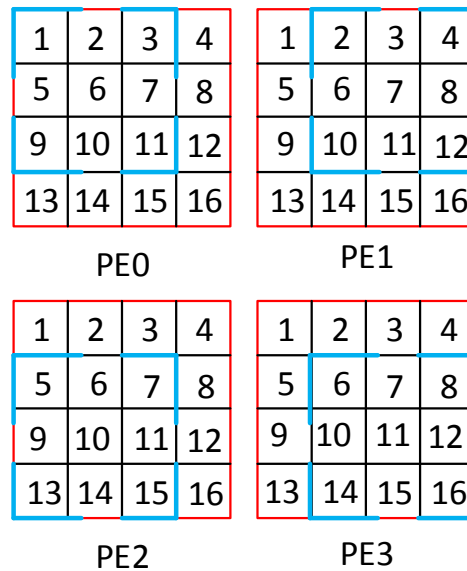
Nguyên lý của bộ Data router để điều khiển các data vào bộ PU như diagram (hình 4.4) sau:



Hình 4.4 Diagram nguyên lý của Data router.

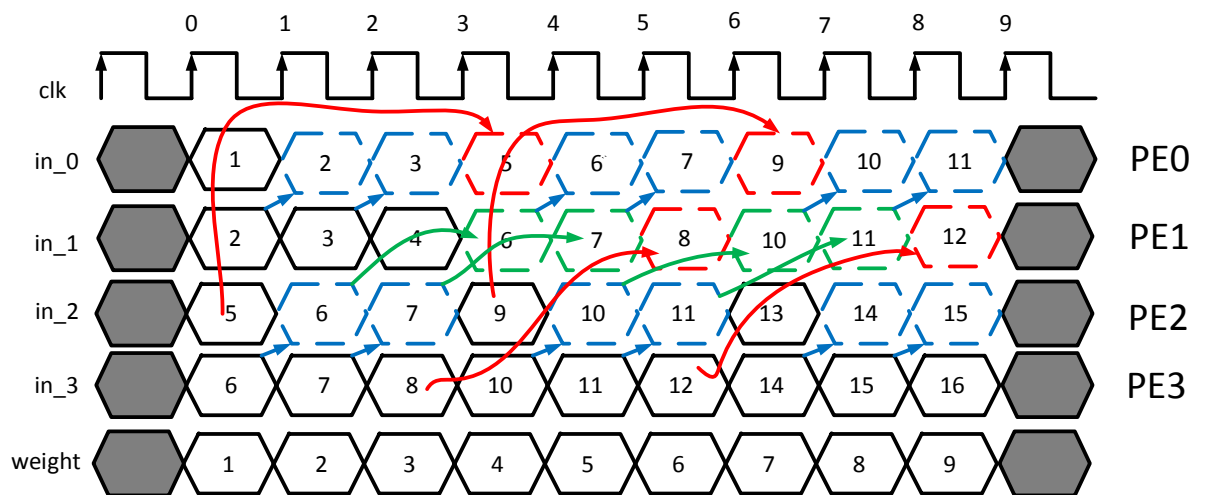
Trong đó  $in_x$  ( $x = 0 \rightarrow 3$ ) là các input data của mỗi PE (mỗi PU có 4 PE), weight là các trọng số.

Một PE sẽ tính toán một pixel của output feature map. Mỗi PE sẽ tính tích chập vùng input màu xanh (3x3) với trọng số tương ứng. Vùng nhớ màu đỏ (4x4) được chia thành 4 vùng màu xanh cho phép tích chập (hình 4.5).



Hình 4.5 cửa sổ trượt box màu xanh duy chuyển dọc theo vùng nhớ là box màu đỏ.

Trong mỗi bộ PU, ngoài 4 PE còn có các bộ FIFO để sử dụng lại dữ liệu cho các lần tính toán ở các chu kỳ tiếp theo. Bên dưới (hình 4.6) là diagram ngõ vào của các PE bao gồm các input feature map và weight. Các PE được thiết kế pipe-lined và cần 9 chu kỳ để tính một phép tích chập.



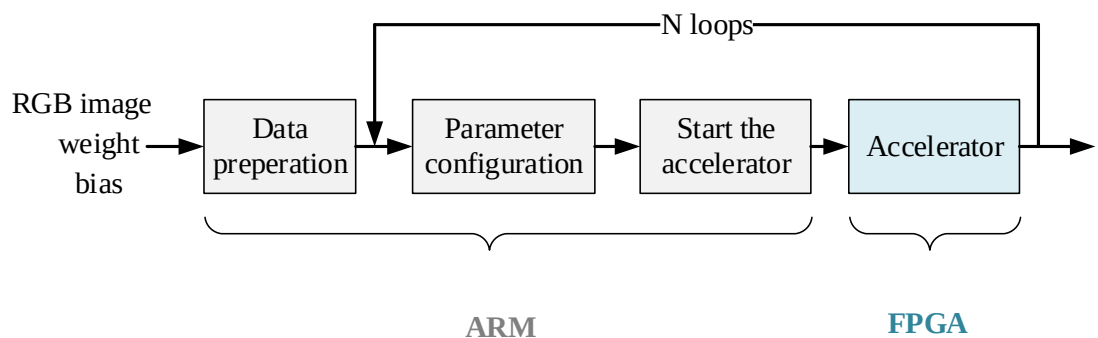
Hình 4.6 Diagram ngõ vào của các PE.

Các data màu đen nét liền là các data được feed vào từ bộ data router, các data màu xanh dương nét đứt là các data ngõ vào của PE bên phải (PE1 bên phải PE0, PE3 bên phải PE2) ở chu kỳ trước đó, các data màu xanh ở PE1 là các data của ngõ

vào PE2 ở 2 chu kì trước đó, các data màu đỏ nét đứt là các data ngõ vào của các PE bên dưới (PE2 bên dưới PE0, PE3 bên dưới PE1) ở 3 chu kỳ trước đó.

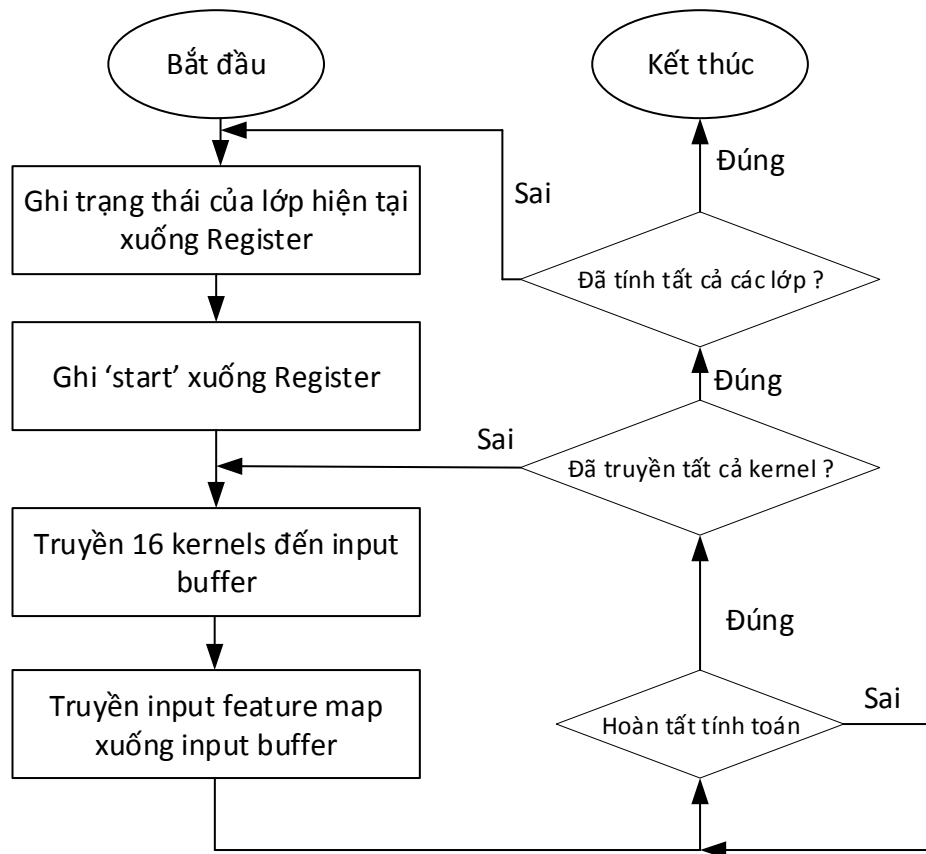
#### 4.2 Phần Mềm Trên Lõi Arm:

Firmware trên lõi ARM đóng vai trò quan trọng trong toàn hệ thống, xử lý các chức năng bao gồm: các hoạt động tiền xử lý như các thao tác trên bộ nhớ: khai báo, khởi tạo, giải phóng các vùng nhớ để lưu ảnh ngõ vào, input, output feature map, trọng số, bias. Khai báo vùng nhớ dùng chung, điều khiển hoạt động của FPGA thông qua việc ghi giá trị vào các thanh ghi trên FPGA. Ta có thể tóm tắt các chức năng của firmware trên ARM thành các phần chính như hình 4.7. Trong đó các khối màu xám sẽ do ARM xử lý, sau bước data preparation thì ARM và FPGA sẽ hoạt động cùng nhau trong một vòng lặp.



Hình 4.7 Vai trò của các thành phần firmware trên ARM.

Các lớp tích chập của mạng CNN được xử lý theo lưu đồ như sau (hình 4.8).



Hình 4.8 Flowchart xử lý các lớp tích chập.

Quá trình này lặp lại đến khi tất cả các lớp tích chập được tính, sau đó thanh ghi DONE sẽ được FPGA set lên 1, Firmware trên ARM quét thấy tín hiệu này sẽ thực hiện tính các lớp Dense còn lại trên CPU và trả về kết quả cuối cùng.

## CHƯƠNG 5: KẾT QUẢ THỰC HIỆN

### 5.1 Cài Đặt Hệ Thống Và Hiện Thực:

Trước tiên, tôi hiện thực mạng VGG16 để đánh giá bộ tăng tốc. Phần cứng được cài đặt từ các công thức (3) tới (6) dựa trên các tính chất của mạng VGG16 và số lượng tài nguyên sẵn có nhằm đạt hiệu quả tối đa. Đối với thiết bị Intel FPGA Cyclone V SoC của tôi, có tất cả 112 DSPs được nhúng trên FPGA và giá trị Nof nhỏ nhất trong tất cả các lớp CONV của mạng VGG16 là 32. Tôi đặt hệ số Cof là 32 để tổng số DSP cần thiết là 128. Các thành phần logic trong FPGA được dùng để xây dựng thêm 16 DSPs còn thiếu. Tôi sử dụng số integer 16 bit để biểu diễn cho cả giá trị điểm ảnh và trọng số. Sau khi xác định được các giá trị cần thiết đặt, tôi dùng Verilog VHDL để mô tả phần cứng. Sau đó, dùng Quartus Prime 16.1 để biên dịch và map vào board DE10-Standard ở tần số xung nhịp 150Mhz. Số lượng tài nguyên đã sử dụng trong thiết kế hệ thống được mô tả trong bảng 5.1.

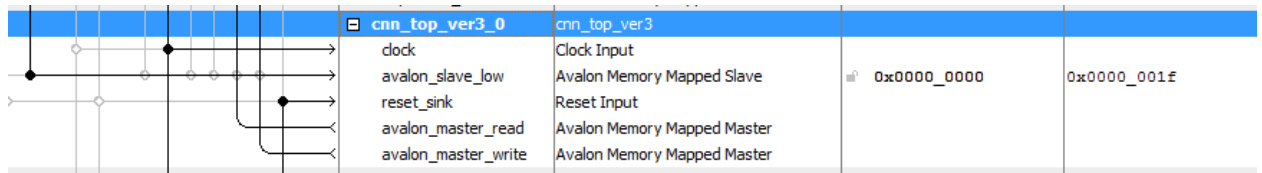
Bảng 5.1 Tài nguyên đã sử dụng.

| Tài nguyên          | Đã dùng     | Có sẵn |
|---------------------|-------------|--------|
| ALMs                | 18964 (45%) | 41509  |
| On-chip memory (Kb) | 4156 (75%)  | 5570   |
| DSPs                | 112 (100%)  | 112    |

Thiết kế của tôi đạt được thông lượng 38,8 GOP / s và độ trễ 0,79 giây / hình ảnh. Con số này khá khác so với tính toán của tôi dựa trên công thức (7), với khoảng 0,5 giây / hình ảnh. Lý do cho điều này xuất phát từ sự sai khác giữa thời gian lý tưởng và thời gian thực tế để truy cập bộ nhớ ngoài.

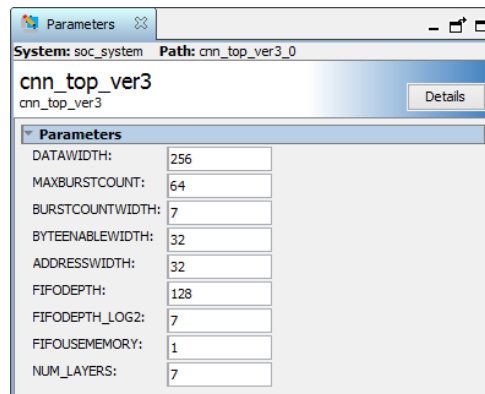
Bộ tăng tốc cho CNN được thiết kế thành lõi IP và tích hợp trở thành 1 thành phần của hệ thống (cnn\_top).

Hình 5.1 thể hiện việc kết nối lõi IP đã thiết kế với các module khác trong Qsys.



Hình 5.1 lõi IP đã thiết kế trong Qsys.

Các tham số thiết lập cho lõi IP tăng tốc (hình 5.2):



Hình 5.2 Bảng thiết lập các tham số cho lõi IP.

## 5.2 So sánh:

Giải pháp của tôi dựa trên các kỹ thuật tương tự như nghiên cứu [9], tuy nhiên được hiện thực trên các nền tảng phần cứng khác nhau. Trong khi [9] dùng các chip Intel FPGA hiệu năng cao thì tôi tận dụng tối đa ưu điểm và tài nguyên của dòng Intel FPGA chi phí thấp. Trong bảng 5.2, bộ tăng tốc của tôi có hệ số hiệu quả tài nguyên (1.6) và hệ số hiệu quả bộ nhớ ( $9.3 \times 10^{-3}$ ) tốt hơn so với [9] (1.6 và  $7.9 \times 10^{-3}$ ). Lý do là tôi chỉ tập trung vào các lớp CONV của mạng VGG do các đặc điểm tương tự của chúng và thực hiện nhiều cải tiến hơn trong nghiên cứu [9] có thể thực hiện trên các mô hình CNN khác nhau cũng như thực hiện tính toán cho cả hai lớp CONV và các lớp Fully-connected.

Bảng 5.2 So sánh với những nghiên cứu khác.

|                 | [9]                        | Ours                          |
|-----------------|----------------------------|-------------------------------|
| Platform        | Intel Stratix V GXA7/28 nm | Intel Cyclone V SX SoC /28 nm |
| Clock (Mhz)     | 150                        | 150                           |
| Precision       | 16-bit fixed point         | 16-bit integer                |
| # of MAC units  | 1568                       | 128                           |
| DSP Utilization | 256                        | 112                           |

|                                     |                      |                                        |
|-------------------------------------|----------------------|----------------------------------------|
| ALMs                                | 218K                 | 19K                                    |
| On-chip memory (Kb)                 | 44200                | 4156                                   |
| Throughput                          | 348.8 (Overall)      | 38.8 (CONV)                            |
| Resource efficiency<br>(GOP/s/ALMs) | 1.6                  | <b>2</b>                               |
| Memory efficiency<br>(GOP/s/Kb)     | $7.9 \times 10^{-3}$ | <b><math>9.3 \times 10^{-3}</math></b> |

## CHƯƠNG 6: SỬ DỤNG LỖI TĂNG TỐC ĐÃ THIẾT KẾ CHO ỨNG DỤNG PHÂN LOẠI CẢM XÚC KHUÔN MẶT

### 6.1 Nhận Diện Cảm Xúc Khuôn Mặt:

Trong phần này, tôi trình bày một hệ thống nhận dạng biểu hiện khuôn mặt (Facial Emotion Recognition hay FER) được triển khai trên thiết bị SoC FPGA. Mục tiêu là tạo ra hệ thống có thể hoạt động độc lập và tự động phân loại 7 loại cảm xúc cơ bản khác nhau là: tức giận, sợ hãi, hạnh phúc, buồn bã, bất ngờ, ghê tởm và trung tính. Tôi dùng lõi IP đã thiết kế trên FPGA để tính toán các lớp mạng thần kinh tích chập sau đó thiết kế một mạng CNN tối ưu với phần cứng được thiết kế để thực hiện nhận dạng cảm xúc và đạt độ chính xác 66% trong bộ dữ liệu FER2013. Toàn bộ hệ thống phần cứng được thiết kế trên SoC FPGA, có thể xử lý 15 khung hình mỗi giây với tần số hoạt động 130 MHz. Tôi chia nhiệm vụ nhận dạng cảm xúc khuôn mặt thành hai giai đoạn. Giai đoạn đầu tiên là phát hiện một khuôn mặt trong đó tôi xây dựng mô-đun nhận diện khuôn mặt chạy trên CPU để cắt vùng 64x64 pixel ở thang xám có chứa khuôn mặt. Giai đoạn thứ hai là dự đoán cảm xúc khuôn mặt cho ảnh trên bằng mô hình CNN.

#### 6.1.1 Bước Phát Hiện Khuôn Mặt:

OpenCV là một thư viện chứa nhiều mô hình được đào tạo trước (pretrained) để phát hiện khuôn mặt, vì vậy tôi đã sử dụng Haar Cascade của OpenCV [26] để phát hiện khuôn mặt vì đây là một trong những thuật toán hiệu quả và phổ biến nhất. Cả Haar Cascade và CNN đều là các thuật toán học máy, nhưng các feature của Haar Cascade được xác định thủ công trong khi các feature của CNN được xác định trong quá trình tối ưu bằng gradient descent. Ưu điểm của nó là tốc độ phát hiện khuôn mặt nhanh và yêu cầu bộ nhớ thấp hơn. Tuy nhiên, nhược điểm của nó trong thực tế là thiếu độ chính xác khi khuôn mặt ở một góc không đối diện với camera và hiệu suất thấp trong khi chạy trên CPU ARM của Cyclone V (dưới 7 khung hình mỗi giây). Do đó, tôi quyết định giảm lượng dữ liệu cần xử lý để cải thiện hiệu suất bằng cách giảm vùng cần xử lý. Khi một khuôn mặt được tìm thấy trong toàn bộ khung hình, tọa độ

vị trí và boundary box của nó được xác định. Sau đó, một vùng quan tâm (Region of Interest - ROI) được tính xung quanh boundary box, vùng này lớn hơn 120% boundary box. Sau đó, một hàm khác được gọi để phát hiện khuôn mặt trong ROI. Do ROI có độ phân giải thấp hơn so với toàn bộ khung hình, quá trình này cải thiện đáng kể tốc độ nhận diện khuôn mặt, thử nghiệm của tôi cho thấy hiệu suất tăng gấp đôi (đạt tối đa 15 khung hình mỗi giây).

### **6.1.2 Bước Phân Loại Cảm Xúc Khuôn Mặt:**

Có một số nghiên cứu trước đây về thiết kế mạng CNN về phân loại cảm xúc khuôn mặt. Một ví dụ điển hình là nghiên cứu của Octavio Arriaga [27], mục tiêu của họ là thiết kế một mạng CNN có thể chạy trong thời gian thực. Kết quả của họ cho thấy kết quả tốt về hiệu suất của mạng mà họ thiết kế, vì vậy thiết kế mạng của tôi thừa hưởng một số thuộc tính từ mạng của họ. Mô hình của tôi gồm bảy lớp chập (hình 6.1), so với mô hình của Octavio Arriaga, mô hình của họ có 9 lớp tích chập với kích thước kernel khác nhau. Tuy nhiên, trong thiết kế của tôi, tôi sử dụng cùng kích thước kernel cho tất cả các lớp chập để giảm độ phức tạp cho thiết kế phần cứng của tôi. Hơn nữa, hệ thống của tôi có ít hơn 2 lớp tích chập để trích xuất các đặc trưng nhưng tôi thêm vào 2 lớp dense vào cuối mạng của mình: lớp thứ nhất có 128 nút và lớp thứ hai có 7 nút để phân loại. Ba lớp max pooling đặt giữa các lớp chập để giảm số lượng tính toán của toàn bộ mạng. Trong giai đoạn huấn luyện, mô hình, tôi sử dụng optimizer là ADAM [28] và Relu làm hàm activation cho mỗi lớp chập. Mô hình được huấn luyện trực tuyến dùng framework Keras và được huấn luyện trên Google Colab để tạo file trọng số.

Sau đó, file trọng số của mô hình được chuyển đổi từ dạng dấu phẩy động ở định dạng hd5 sang số nguyên 16 bit ở định dạng văn bản text .txt. Các tệp văn bản này được lưu trữ trong thẻ SD và sẽ được sử dụng làm đầu vào cho FPGA sau này.



Hình 6.1 Kiến trúc mạng CNN được thiết kế.

## 6.2 Thiết Kế Hệ Thống:

### 6.2.1 Tổng Quan Kiến Trúc:

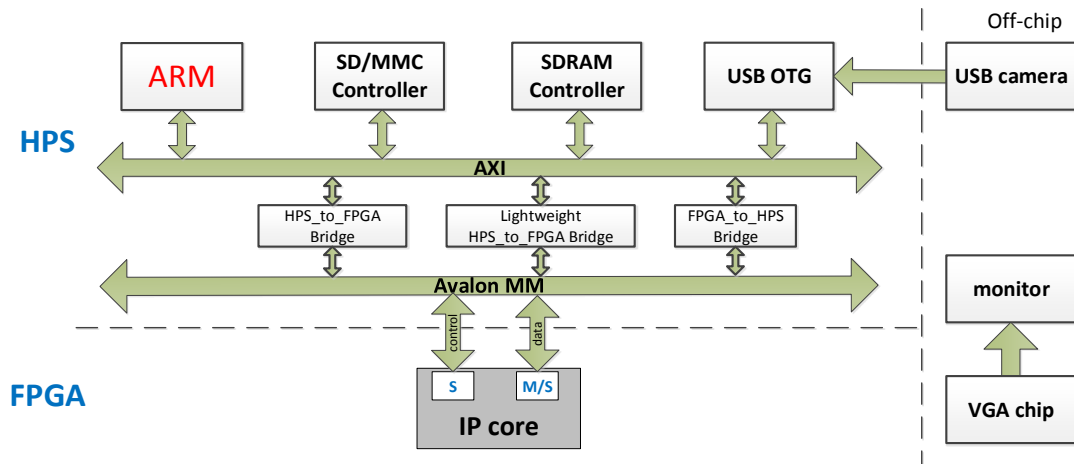
Thiết bị Cyclone V SoC cung cấp hai loại tài nguyên có thể lập trình: CPU ARM và FPGA. Do đó, tôi xây dựng một kiến trúc không đồng nhất như trong hình 6.2 để tận dụng lợi thế linh hoạt của CPU và hiệu năng tính toán cao của FPGA. Do đó, hệ thống của tôi bao gồm hai phần bao gồm phần HPS và phần FPGA.

HPS chịu trách nhiệm cho giai đoạn phát hiện khuôn mặt và điều khiển hoạt động của bộ tăng tốc trên FPGA.

Phần FPGA chứa bộ tăng tốc tích hợp của tôi dưới dạng lõi IP kết hợp với Avalon MM để giao tiếp với các thành phần khác trong hệ thống. Nó chịu trách nhiệm xử lý các lớp chập, max-pooling và Relu. Lõi chứa bộ xử lý trung tâm Process Engine (PE), DMA, bộ đệm dữ liệu ngõ vào và ra (Input and Output buffer), bộ điều khiển Avalon MM.

FPGA và HPS giao tiếp với nhau thông qua ba cầu nối: HPS đến FPGA, Lightweight HPS tới FPGA và FPGA với HPS. Chúng là những cầu AXI được sử dụng để kết nối bus AXI và bus Avalon, những bus này mang dữ liệu và lệnh điều khiển giữa HPS và FPGA.

Trước khi bắt đầu xử lý, HPS lưu trữ hình ảnh đầu vào và các trọng số được sắp xếp từ trước, đã được lượng tử hóa vào SDRAM, đây là quá trình ban đầu.



Hình 6.2 Kiến trúc không đồng nhất giữa CPU và FPGA.

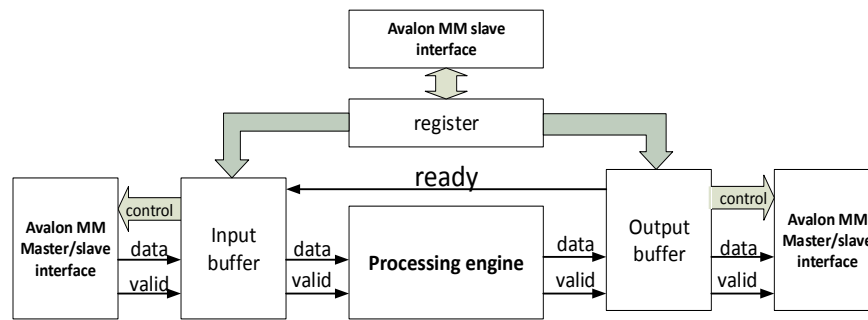
### 6.2.2 Kiến Trúc HPS:

Phần HPS trong hệ thống bao gồm: CPU ARM lõi kép, bộ nhớ ngoài, ba cầu nối AXI, bộ điều khiển SDRAM, USB OTG và bộ điều khiển SD. Bộ nhớ ngoài được cả HPS và FPGA sử dụng. Bộ xử lý ARM xử lý dữ liệu video từ camera để cung cấp hình ảnh khuôn mặt cho FPGA. HPS cũng tính toán các lớp dense và cung cấp môi trường để hiển thị kết quả phát sau khi xử lý trên màn hình LCD. Khối USB OTG chịu trách nhiệm giao tiếp giữa hệ thống của tôi và camera USB Logitech. Khối điều khiển SDRAM chịu trách nhiệm kiểm soát truy cập bộ nhớ ngoài.

### 6.2.3 Kiến trúc lõi tăng tốc:

Kiến trúc mạng thần kinh của tôi được lấy cảm hứng từ kiến trúc VGG, các thuộc tính lặp đi lặp lại trong việc sử dụng các bộ lọc nhỏ có kích thước 3x3 của các lớp chập rất thuận tiện cho việc hiện thực trên FPGA. Do đó, bộ tính toán cho kích thước kernel 3x3 có thể sử dụng cho tất cả các lớp chập. Do tài nguyên của FPGA hạn chế, đặc biệt là số lượng DSP, điều này dẫn đến việc tôi không thực hiện tính lớp dense trên FPGA và để lại các phép tính này cho CPU xử lý. IP đã được thiết kế của tôi mô tả trong hình 6.3. Nó bao gồm hai bộ đệm cho dữ liệu đầu vào và đầu ra, thanh ghi, giao tiếp Avalon MM và bộ xử lý trung tâm Process Engine (PE).

Bộ đệm đầu vào được sử dụng để lưu trữ các tham số của mạng CNN, như input feature map, bias và trọng số trước khi các dữ liệu này được xử lý bởi PE. Bộ đệm đầu ra được sử dụng để lưu trữ dữ liệu được trả về từ PE.



Hình 6.3 Kiến trúc bộ tăng tốc.

Thành phần tính toán cốt lõi trong thiết kế là PE, nó được xây dựng từ 112 DSP và chịu trách nhiệm tính toán tích chập 2D từ các input feature map và các trọng số sau đó trả về các output feature map. Ngoài ra, PE có thể tính toán max pooling và RELU.

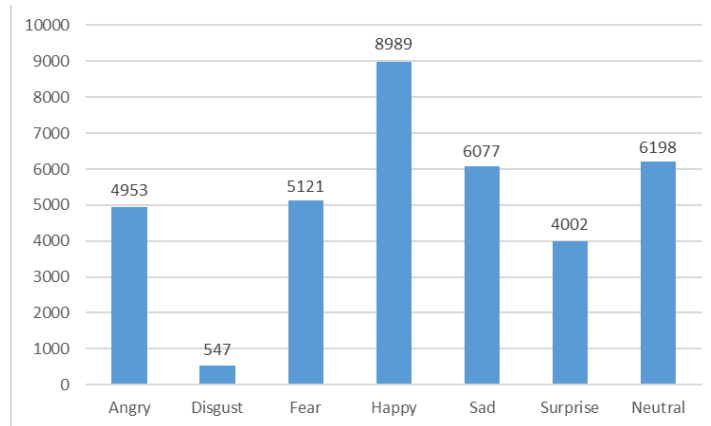
Để kiểm soát hoạt động của lõi IP, tôi sử dụng 5 thanh ghi, những thanh ghi này có thể được truy cập bởi cả CPU và IP. 5 thanh ghi này có các chức năng sau: Thanh ghi đầu tiên được sử dụng để cho phép PE hoạt động. Thanh ghi thứ hai thông báo cho PE rằng quá trình xử lý đã hoàn tất. Thanh ghi thứ ba chứa thông tin về khối lượng tích chập đang được xử lý bao gồm kích thước của các input feature map, số bản input feature map, số bản output feature map và có hay không việc tính max-pooling. Hai thanh ghi cuối cùng được sử dụng để lưu trữ địa chỉ của bộ nhớ ngoài để đọc và ghi dữ liệu.

Để giao tiếp với các thành phần khác trong hệ thống SoC, tôi sử dụng giao tiếp Avalon MM, bao gồm 2 mô-đun: master và slave. Mô-đun master hiệu suất cao được sử dụng để truyền các input feature map, trọng số và output feature map giữa IP và bộ nhớ ngoài. Trong khi đó mô-đun slave được sử dụng để giao tiếp với CPU ARM.

### 6.3 Thử Nghiệm Và Kết Quả:

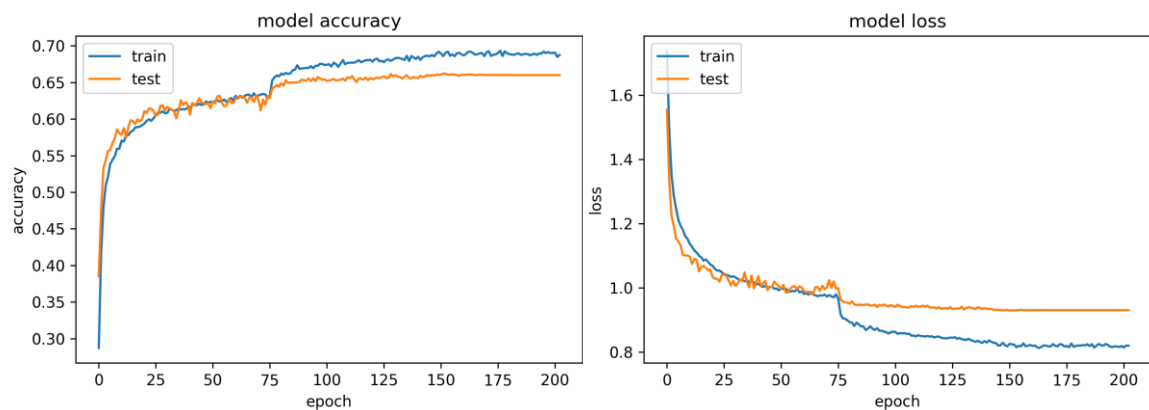
Tôi đã lấy tập dữ liệu phổ biến là FER-2013 [29], bộ dữ liệu về cảm xúc khuôn mặt để huấn luyện và đánh giá mô hình CNN của tôi. Bộ dữ liệu này được tạo bởi Aaron Courville và Pierre-Luc Carrie. Bộ dữ liệu có bảy loại (hoặc lớp) biểu cảm khuôn mặt: tức giận, sợ hãi, hạnh phúc, buồn bã, bất ngờ, ghê tởm và trung tính với tổng số 35,487 ảnh xám kích thước 48 x 48 pixel. Các hình ảnh trong mỗi lớp không

được phân phối đồng đều, chi tiết như trong hình 6.4. Những hình ảnh này được dán nhãn theo đánh giá của con người và trên bộ dữ liệu này, độ chính xác của con người là  $65 \pm 5\%$ .



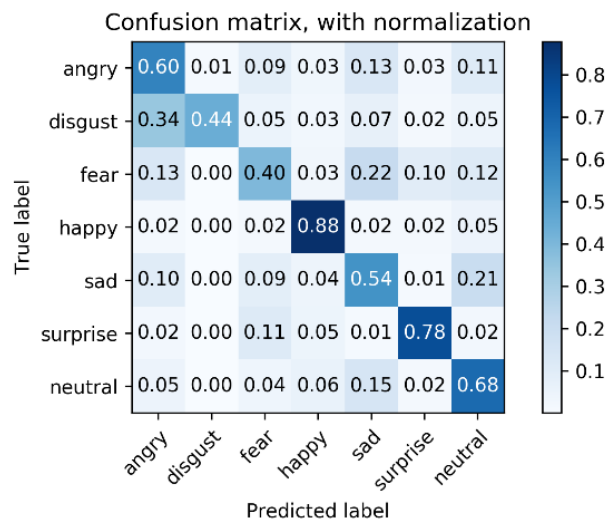
Hình 6.4 Phân bố hình ảnh trong các lớp.

Trong giai đoạn huấn luyện trên Google Colab, mục tiêu của tôi là huấn luyện mạng thần kinh có khả năng tự phân loại bảy cảm xúc. Trong quá trình huấn luyện, optimizer Adam được sử dụng, sau 200 epoch, độ chính xác đạt 66% đối với tập dữ liệu validate và 69% đối với tập dữ liệu train mà không có hiện tượng overfitting. Đây là kết quả của việc sử dụng kỹ thuật Drop-out [30] trước các lớp dense trong mạng thần kinh. Lịch sử chi tiết của quá trình huấn luyện của tôi như trong hình 6.5.



Hình 6.5 Trái: độ chính xác của mô hình, phải: loss của mô hình.

Mặc dù kiến trúc của tôi có ít lớp tích chập hơn và chỉ sử dụng một kích thước kernel, tôi vẫn đạt được độ chính xác tương tự như kết quả của Octavio Arriaga [27], mô hình của họ đạt được độ chính xác tới 66% khi thử nghiệm trên bộ dữ liệu FER-2013.



Hình 6.6 Confusion matrix khi thử nghiệm trên tập dữ liệu FER2013.

Hình 6.6 là confusion matrix cho mô hình của tôi khi được thử nghiệm trên FER2013, trong đó hạnh phúc có tỷ lệ nhận dạng cao nhất là 88% trong khi sợ hãi có tỷ lệ nhận dạng thấp nhất là 40%. Lý do rất có thể là biểu cảm khuôn mặt đáng sợ có nhiều điểm tương đồng với nét mặt buồn.

Trong nghiên cứu này, tôi đã sử dụng Quartus phiên bản 16.1 để tổng hợp thiết kế RTL cho board DE-10 standard. Sau khi tổng hợp thiết kế đã sử dụng hầu hết ba loại tài nguyên phần cứng: ALM, bộ xử lý tín hiệu số (DPS) và các khối RAM. Thiết kế đã tiêu thụ tất cả các khối DSP và RAM và một nửa ALM của Cyclone V. Báo cáo chi tiết về Quartusfime của tài nguyên đã sử dụng được trình bày trong bảng sau (bảng 6.1 ).

Bảng 6.1 Tài nguyên đã sử dụng trong thiết kế.

| Type of resource | <b><i>Proposed system</i></b> | <b><i>FPGA resources</i></b> |
|------------------|-------------------------------|------------------------------|
| ALMs             | 22,465 (54%)                  | 41,910                       |
| DSPs             | 112 (100%)                    | 112                          |
| RAM blocks       | 553 (100%)                    | 553                          |
| PLLs             | 1 (7%)                        | 15                           |
| DLLs             | 1 (25%)                       | 4                            |

Lỗi tăng tốc trên FPGA có thể hoạt động ở mức 130 MHz, mỗi hình ảnh khuôn mặt 64x64 được phân loại trong 6,37ms (157 khuôn mặt mỗi giây).

So với nghiên cứu tương tự của Hanh Phan-Xuan và các cộng sự [31], mô hình của tôi đạt được độ chính xác 66% trong khi báo cáo của họ cho thấy độ chính xác đạt 60,3%. Bù đắp cho độ chính xác cao hơn, kiến trúc mạng nơron tích chập của tôi sâu hơn so với họ, điều này dẫn đến hình ảnh mất nhiều thời gian hơn để được phân loại. Các chi tiết so sánh được trình bày trong bảng 6.2 dưới đây.

Bảng 6.2 So sánh với nghiên cứu khác.

|                  | <i>Hanh et al. [31]</i> | <i>Our</i> |
|------------------|-------------------------|------------|
| Accuracy         | 60.3%                   | 66%        |
| DSPs             | 152                     | 112        |
| Clock frequency  | 200 Mhz                 | 130 Mhz    |
| Faces per second | 400                     | 157        |



Hình 6.7 Hình ảnh demo hệ thống.

Hình 6.7 cho thấy kết quả thử nghiệm hệ thống của tôi. Như chúng ta có thể thấy, hệ thống có khả năng phát hiện và phân loại chính xác khuôn mặt của tác giả với tốc độ trung bình 15 khung hình mỗi giây. Tôi đã tải lên video demo trên Youtube tại: <https://youtu.be/V7AFTmO5RUA>

Nghiên cứu này đã được công bố tại hội nghị ISEE 2019 và xuất bản trên IEEE tại địa chỉ:

<https://ieeexplore.ieee.org/document/8921140>

## CHƯƠNG 7: KẾT LUẬN

Nghiên cứu này đề xuất bộ tăng tốc cho các lớp VGG CONV được triển khai trên bo mạch Intel Cyclone V SoC. Bằng cách cấu hình lại các tham số như kích thước của feature map, số lượng feature map đầu vào và đầu ra thông qua phần mềm, máy gia tốc có thể hỗ trợ cho bất kỳ mạng nào giống VGG. Bộ tăng tốc được tích hợp trong một hệ thống không đồng nhất, sử dụng RAM DDR3 tốc độ cao làm bộ nhớ dùng chung. Băng thông của bộ nhớ ngoài được cải thiện lên tới 4,6GB / giây, giúp tăng đáng kể hiệu suất truyền dữ liệu. Sau đó, nó được đánh giá trên mạng rất lớn là VGG16 và đạt được thông lượng 38,8 GOP / s dưới tần số làm việc 150 MHz cho tất cả các lớp CONV. Sau đó, sử dụng lõi đã thiết kế cho một hệ thống độc lập có thể ghi dữ liệu video từ camera USB để phát hiện khuôn mặt người và nhận biết cảm xúc của họ trên một thiết bị SoC giá rẻ. Mạng CNN đã được thiết kế mang lại độ chính xác tốt trong việc phân loại cảm xúc và hoạt động hiệu quả trên phần cứng. IP được thiết kế trên FPGA có thể cung cấp hiệu suất cao trong ứng dụng thực tế với tốc độ 157 hình ảnh được phân loại mỗi giây khi sử dụng với mạng CNN đã được thiết kế. Tốc độ này đáp ứng được các yêu cầu xử lý trong thời gian thực. Tuy nhiên, toàn bộ hệ thống chỉ đạt được tốc độ 15 khung hình mỗi giây do giới hạn tốc độ của CPU ARM trên SoC FPGA.

Đây là lý do tại sao các nghiên cứu trong tương lai, tôi sẽ giảm số lượng tính toán mà CPU cần xử lý bằng cách chuyển giai đoạn phát hiện khuôn mặt sang FPGA. Một lựa chọn khác là kết nối thêm các luồng dữ liệu hình ảnh mới cho các GPU để tiến hành phân loại, khi đó hệ thống sẽ là một cấu trúc topo dạng sao với FPGA là một thành phần trung tâm. Bằng cách giảm bớt các tắc nghẽn về hiệu suất ở CPU, tôi hi vọng sẽ cải thiện đáng kể hiệu năng tổng thể của hệ thống.

## TÀI LIỆU THAM KHẢO

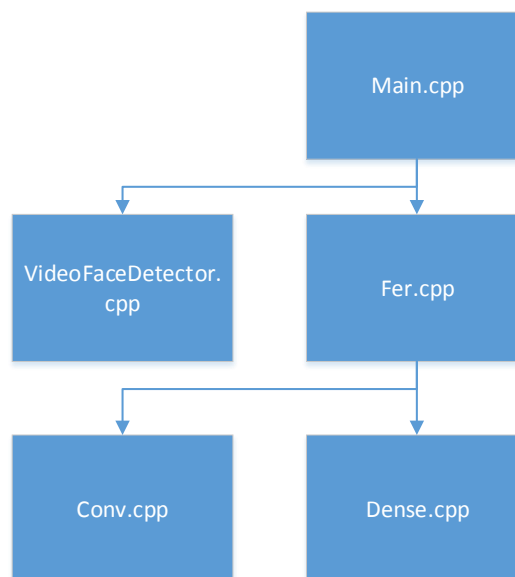
- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in neural information processing systems*, 2012, pp. 1097--1105.
- [2] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *CoRR*, vol. abs/1409.1556, 2014.
- [3] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. A. Alemi, "Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning," in *Proceedings of the 31st AAAI Conference on Artificial Intelligence*, 2017, pp. 4278--4284.
- [4] M. Rezaee, Y. Zhang, R. Mishra, F. Tong and H. Tong, "Using a VGG-16 Network for Individual Tree Species Detection with an Object-Based Approach," *2018 10th IAPR Workshop on Pattern Recognition in Remote Sensing (PRRS)*, Beijing, 2018, pp. 1-7.
- [5] Y. Wu, X. Qin, Y. Pan and C. Yuan, "Convolution Neural Network based Transfer Learning for Classification of Flowers," *2018 IEEE 3rd International Conference on Signal and Image Processing (ICSIP)*, Shenzhen, 2018, pp. 562-566.
- [6] S. K. Ramani, "Facial Expression Detection Using Neural Network for Customer Based Service," *2018 International Conference on Computer, Communication, and Signal Processing (ICCCSP)*, Chennai, 2018, pp. 1-4.
- [7] NVIDIA Corporation, "NVIDIA cuDNN GPU accelerated deep learning," <https://developer.nvidia.com/cudnn>
- [8] J. Qiu et al., "Going Deeper with Embedded FPGA Platform for Convolutional Neural Network.," in *FPGA*, 2016, pp. 26--35.
- [9] Y. Ma, Y. Cao, S. B. K. Vrudhula, and J. sun Seo, "Optimizing the Convolution Operation to Accelerate Deep Neural Networks on FPGA.," *IEEE Trans. VLSI Syst.*, vol. 26, no. 7, pp. 1354--1367, 2018.
- [10] Y. Chen, T. Krishna, J. S. Emer and V. Sze, "Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks," in *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127-138, Jan. 2017.
- [11] R. DiCecco, G. Lacey, J. Vasiljevic, P. Chow, G. Taylor and S. Areibi, "Caffeinated FPGAs: FPGA framework For Convolutional Neural Networks," *2016 International Conference on Field-Programmable Technology (FPT)*, Xi'an, 2016, pp. 265-268.
- [12] C. Zhang and V. K. Prasanna, "Frequency Domain Acceleration of Convolutional Neural Networks on CPU-FPGA Shared Memory System.," in *FPGA*, 2017, pp. 35--44.
- [13] X. Wei et al., "Automated Systolic Array Architecture Synthesis for High Throughput CNN Inference on FPGAs.," in *DAC*, 2017, p. 29:1-29:6.
- [14] C. Szegedy et al., "Going deeper with convolutions," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 1--9.
- [15] N. Suda et al., "Throughput-Optimized OpenCL-based FPGA Accelerator for Large-Scale Convolutional Neural Networks.," in *FPGA*, 2016, pp. 16--25.
- [16] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing FPGA-based Accelerator Design for Deep Convolutional Neural Networks.," in *FPGA*, 2015, pp. 161--170.
- [17] L. Du et al., "A Reconfigurable Streaming Deep Convolutional Neural Network Accelerator for Internet of Things.," *IEEE Trans. on Circuits and Systems*, vol. 65--I, no. 1, pp. 198--208, 2018.
- [18] A. Rahman, J. Lee, and K. Choi, "Efficient FPGA acceleration of Convolutional Neural Networks using logical-3D compute array.," in *DATE*, 2016, pp. 1393--1398.
- [19] P. Ekman and W. Friesen, "Constants across cultures in the face and emotion.," *Journal of Personality and Social Psychology*, vol. 17, no. 2, pp. 124-129, 1971. Available: 10.1037/h0030377.
- [20] Y. LeCun, Y. Bengio and G. Hinton, "Deep learning", *Nature*, vol. 521, no. 7553, pp. 436-444, 2015. Available: 10.1038/nature14539.
- [21] Y. Ma, Y. Cao, S. B. K. Vrudhula, and J. sun Seo, "Optimizing the Convolution Operation to Accelerate Deep Neural Networks on FPGA.," *IEEE Trans. VLSI Syst.*, vol. 26, no. 7, pp. 1354--1367, 2018.
- [22] J. Wang, J. Lin, and Z. Wang, "Efficient Hardware Architectures for Deep Convolutional Neural Network.," *IEEE Trans. on Circuits and Systems*, vol. 65--I, no. 6, pp. 1941--1953, 2018.
- [23] J. Qiu et al., "Going Deeper with Embedded FPGA Platform for Convolutional Neural Network.," in *FPGA*, 2016, pp. 26--35.
- [24] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.

- [25] C. Szegedy et al., “*Going deeper with convolutions*,” in Proceedings of the IEEE conference on computer vision and pattern recognition, 2015, pp. 1--9.
- [26] P. A. Viola and M. J. Jones, “*Rapid Object Detection using a Boosted Cascade of Simple Features.*,” in CVPR (1), 2001, pp. 511–518.
- [27] O. Arriaga, M. Valdenegro-Toro, and P. Plöger, “*Real-time Convolutional Neural Networks for Emotion and Gender Classification.*,” CoRR, vol. abs/1710.07557, 2017.
- [28] D. P. Kingma and J. Ba, “*Adam: A method for stochastic optimization*,” arXiv preprint arXiv:1412.6980, 2014.
- [29] I. Goodfellow et al., “*Challenges in representation learning: A report on three machine learning contests*”, Neural Networks, vol. 64, pp. 59-63, 2015. Available: 10.1016/j.neunet.2014.09.005.
- [30] N. Srivastava, G. E. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “*Dropout: a simple way to prevent neural networks from overfitting.*,” Journal of Machine Learning Research, vol. 15, no. 1, pp. 1929-1958, 2014.
- [31] Hanh Phan-Xuan, ThuongLe-Tien, SyNguyen-Tan “*FPGA Platform applied for Facial Expression Recognition System using Convolutional Neural Networks*”, Procedia Computer Science Volume 151, 2019, Pages 651-658.

## CHƯƠNG 8: PHỤ LỤC

Trong phụ lục này sẽ trình bày firmware của ứng dụng phân loại cảm xúc khuôn mặt với hình ảnh được thu từ camera và dùng bộ tăng tốc để thực hiện tính toán các lớp tích chập. Hình 7.1 mô tả cấu trúc của firmware trên ARM, mỗi module có vai trò và chức năng riêng biệt trong hệ thống, trong đó:

- Module main là module chính của firmware có chức năng khởi tạo hệ thống, thu thập dữ liệu đầu vào từ camera chuyển cho module VideoFaceDetector để phát hiện khuôn mặt, khuôn mặt đã được phát hiện sẽ chuyển cho module Fer để phân loại cảm xúc. Cuối cùng là hiển thị kết quả ra LCD.
- Module VideoFaceDetector đảm nhận vai trò phát hiện khuôn mặt.
- Module Fer đảm nhận vai trò phân loại cảm xúc khuôn mặt, điều khiển hành vi và dữ liệu giữa 2 thành phần HPS và FPGA thông qua 2 module con là: Conv và Dense.
- Module Conv có nhiệm vụ tính các lớp tích chập, điều khiển hành vi và dữ liệu cho bộ tăng tốc trên FPGA.
- Module Dense có nhiệm vụ tính các lớp dense sau khi các lớp tích chập được tính sau, thực hiện trên CPU.



Hình 8.1 Cấu trúc firmware trên ARM.

## Nội dung file main.c

```

#include <opencv2/highgui.hpp>
#include <opencv2/imgproc.hpp>
#include "VideoFaceDetector.h"
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include "hwlib.h"
#include "socal/socal.h"
#include "socal/hps.h"
#include "socal/alt_gpio.h"
// #include "hps_0.h"
#include <omp.h>
#include <sys/time.h>
#include <time.h>
#include "fer.h"
#include <string>
#include <iostream>
#include <sstream>
#include <iomanip>
#define HW_REGS_BASE ( ALT_STM_OFST )
#define HW_REGS_SPAN ( 0x04000000 )
#define HW_REGS_MASK ( HW_REGS_SPAN - 1 )
// using namespace cv ;
using namespace std ;
static long get_tick_count(void)
{
    int k;
    struct timespec now;
    k = clock_gettime(CLOCK_MONOTONIC, &now);
    printf("suc %d\n",k);
    return now.tv_sec*1000000 + now.tv_nsec/1000;
}

const cv::String WINDOW_NAME("Camera video");
const cv::String WINDOW_NAME2("Camera video 2");
// file xml có sẵn của opencv
const cv::String CASCADE_FILE("haarcascade_frontalface_default.xml");
int main(int argc, char** argv)
{
    // Khởi tạo hệ thống và bộ nhớ
    void *virtual_base;
    void *h2p_low_cnn_addr;
    unsigned long *mem_addr;
    unsigned long time;
    float *flatten_base;
    unsigned long out_base;
    int fd;
    unsigned long *mem;
    float acc;
    stringstream stream;
    string fps_s;
    flatten_base = new float [128*4*4];
    if(flatten_base == NULL)
    {
        printf("cannot alloc fl mem\n");
        return 0;
    }
    if( ( fd = open( "/dev/mem", ( O_RDWR | O_SYNC ) ) ) == -1 ) {
        printf( "ERROR: could not open \"/dev/mem\"...\n" );
        return( 1 );
    }
    virtual_base = mmap( NULL, HW_REGS_SPAN, ( PROT_READ | PROT_WRITE ), MAP_SHARED,
fd, HW_REGS_BASE );

    if( virtual_base == MAP_FAILED ) {
        printf( "ERROR: mmap() failed...\n" );
        close( fd );
        return( 1 );
    }
}

```

```

    h2p_low_cnn_addr=virtual_base + ( ( unsigned long ) ( ALT_LWFGASLVS_OFST +0x00
) & ( unsigned long)( HW_REGS_MASK ) );
    mem_addr = ( unsigned long *)mmap( NULL, 0x8000000, ( PROT_READ | PROT_WRITE ),
MAP_SHARED, fd,0x30000000 );
    if( mem_addr == MAP_FAILED ) {
        printf( "ERROR: mem mmap() failed...\n" );
        close( fd );
        return( 1 );
    }
    mem = ( unsigned long *)mmap( NULL, 0x1000, ( PROT_READ | PROT_WRITE ),
MAP_SHARED, fd,0xffc25000 );
    if( mem == MAP_FAILED ) {
        printf( "ERROR: mem mmap() failed...\n" );
        close( fd );
        return( 1 );
    }
    mem[0x80>>2] = 0x3fff;
    printf("%.8x\n",mem[0x80>>2]);
    printf("Initialize memory map done\n");
    init_mem_conv(mem_addr);
    printf("Initialize convolution memory done\n");
    init_memory();
    printf("Initialize memory dense done\n");
    read_weights();
    printf("read weight dense done\n");

    string s;
    cv::Size size(64,64);
    // Mở camera
    cv::VideoCapture camera(0);
    camera.set(CV_CAP_PROP_FRAME_WIDTH, 640);
    camera.set(CV_CAP_PROP_FRAME_HEIGHT, 480);
    if (!camera.isOpened()) {
        fprintf(stderr, "Error getting camera...\n");
        exit(1);
    }
    cv::namedWindow(WINDOW_NAME, cv::WINDOW_KEEPRATIO | cv::WINDOW_AUTOSIZE);
    cv::namedWindow(WINDOW_NAME2, cv::WINDOW_KEEPRATIO | cv::WINDOW_AUTOSIZE);
    //tìm khuôn mặt
    VideoFaceDetector detector(CASCADE_FILE, camera);
    cv::Mat frame ;
    cv::Mat greyMat;
    cv::Mat croppedImage;
    double fps = 0, time_per_frame;
    //vòng lặp chính, tìm khuôn mặt và dự đoán cảm xúc liên tục
    while (true)
    {
        int start = cv::getCPUTickCount();
        detector >> frame;
        //nếu khuôn mặt đã được tìm thấy thì tính ROI xung quanh khuôn mặt đó để tăng hiệu suất
        if (detector.isFaceFound())
        {
            cv::rectangle(frame, detector.face(), cv::Scalar(255, 0, 0));
            cv::Rect myROI = detector.face();
            greyMat = frame(myROI);
            cvtColor(greyMat, greyMat, CV_BGR2GRAY);
            cv::resize(greyMat, greyMat, size);
            //phân loại cảm xúc khuôn mặt bằng hàm fer_predict, tăng tốc bằng FPGA
            int result = fer_predict(greyMat,mem_addr,(unsigned long
*)h2p_low_cnn_addr,flatten_base,&acc);
            int end = cv::getCPUTickCount();
            time_per_frame = (end - start) / cv::getTickFrequency();
            fps = (15 * fps + (1 / time_per_frame)) / 16;
            printf("Time per frame: %3.3f\tFPS: %3.3f\n", time_per_frame,
fps+2);

            switch (result)
            {
                case 0:
                {
                    s ="angry";
                    break;
                }
                case 1:
                {

```

```

                s ="disgust";
                break;
            }
            case 2:
            {
                s ="fear";
                break;
            }
            case 3:
            {
                s ="happy";
                break;
            }
            case 4:
            {
                s ="sad";
                break;
            }
            case 5:
            {
                s ="surprise";
                break;
            }
            case 6:
            {
                s ="neutral";
                break;
            }
            default:
                break;
        }
        cout << "s = " <<s<< " " <<"acc = "<< acc << endl;
        cv::putText(frame,s,cv::Point2f(10,200), cv::FONT_HERSHEY_DUPLEX,
2, cv::Scalar(0,0,255,255),2);
    }

    cv::imshow(WINDOW_NAME, frame);
    if (cv::waitKey(25)==27) break;
}
// thoát vòng lặp chính, giải phóng bộ nhớ
free_memory();
if( munmap( virtual_base, HW_REGS_SPAN ) != 0 ) {
    printf( "ERROR: munmap() failed...\n" );
    close( fd );
    return( 1 );
}

if( munmap( mem_addr, 0x8000000 ) != 0 ) {
    printf( "ERROR: munmap() failed...\n" );
    close( fd );
    return( 1 );
}

if( munmap( mem, 0x1000 ) != 0 ) {
    printf( "ERROR: munmap() failed...\n" );
    close( fd );
    return( 1 );
}
close( fd );
return 0;
}

```

Nội dung file fer.h

```

#ifndef __FER_H__
#define __FER_H__
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <omp.h>
#include <stdint.h>
#include "conv.h"
#include "dense.h"

```

```
#include <opencv2/highgui.hpp>
#include <opencv2/imgproc.hpp>
int fer_predict(cv::Mat &frame,unsigned long *addr_mem,unsigned long * low_cnn_addr,float
*flatten_base,float *acc);
#endif
```

#### Nội dung file fer.cpp

```
#include "fer.h"
//hàm phân loại cảm xúc khuôn mặt
int fer_predict(cv::Mat &frame,unsigned long *addr_mem,unsigned long * low_cnn_addr,float
*flatten_base,float *acc)
{
    int x,y,k=0;
    unsigned long addr=0;

    for(y=0;y<frame.rows+2;y+=2)
    {
        for(x=0;x<frame.cols+2;x++)
        {
            if(x==0 )
            {
                ddr_mem[addr++] = 0;
                ddr_mem[addr++] = 0;
                ddr_mem[addr++] = 0;
            }
            else if( x == (frame.cols+1))
            {
                for(int i=0;i<45;i++)
                    ddr_mem[addr++] = 0;
            }
            else
            {
                if(y==0)
                {
                    int color = (int)frame.at<uchar>(y,x-1);
                    int a = color-128;
                    ddr_mem[addr++] = a&0xffff;
                    ddr_mem[addr++] = a&0xffff;
                    ddr_mem[addr++] = a&0xffff;
                }
                else if(y == frame.rows)
                {
                    int color = (int)frame.at<uchar>(y-1,x-1);
                    int a = color-128;
                    ddr_mem[addr++] = a<<16;
                    ddr_mem[addr++] = a<<16;
                    ddr_mem[addr++] = a<<16;
                }
                else
                {
                    int color1 = (int)frame.at<uchar>(y-1,x-1);
                    int color2 = (int)frame.at<uchar>(y,x-1);
                    int a = color1-128;
                    int b = color2-128;
                    ddr_mem[addr++] = a<<16 | (b&0x0000ffff);
                    ddr_mem[addr++] = a<<16 | (b&0x0000ffff);
                    ddr_mem[addr++] = a<<16 | (b&0x0000ffff);
                }
            }
        }
    }

    //gọi hàm tính các lớp tích chập từ module Conv, kết quả trả về được lưu vào vùng nhớ
    flatten_base
    vgg16_convolution (low_cnn_addr,addr_mem,flatten_base);
    //gọi hàm tính dense từ module Dense, nạp dữ liệu từ vùng nhớ flatten base ở trên
    //acc: giá trị của cảm xúc tự tin nhất mà mạng dự đoán (lớn nhất của lớp dense cuối cùng)
    k = get_VGG16_predict(flatten_base,acc);
    return k;
}
```

#### Nội dung file Conv.h

```

#ifndef __CONV_H__
#define __CONV_H__
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <omp.h>
#include <stdint.h>
#include <unistd.h>
//#include "weights.h"

#define NUM_LAYER 7

extern int cshape[NUM_LAYER][4];
extern char weight_name[7][50];
extern char bias_name[7][50];

void write_img (const char *file_img,int insize,int nchannel,unsigned long *ddr_addr);
void write_start(unsigned long * low_cnn_addr);
void implement_a_layer(int pooling,int insize,int nkernel,int nchannel,unsigned long *
low_cnn_addr,unsigned long * ddr_addr,int layer,uint32_t rbase,uint32_t wbase);
void flatten (int16_t *base_in, float *base_out);
void vgg16_convolution (unsigned long * low_cnn_addr,unsigned long * ddr_addr,float
*flatten_base);
void write_bias (FILE *file_bs,int nkernel,unsigned long *ddr_addr);
void write_weight(FILE* file_we, int nchannel,int nkernel,unsigned long *ddr_addr);
void init_mem_conv(unsigned long * ddr_addr);
#endif

```

#### Nội dung file Conv.cpp

```

#include "conv.h"
//khai báo kích thước các lớp tích chập, giá trị 0 hoặc 1 cuối cùng là có hay không có
lớp max-pooling theo sau
int cshape[NUM_LAYER][4] = {
    { 32, 3, 64 ,0},
    { 32, 32, 64 ,1},
    { 32, 32, 32 ,0},
    { 32, 32,32, 1},
    { 64, 32,16 ,0},
    { 64, 64, 16,1},
    { 128, 64, 8 ,1}
};
//7 file trọng số của 7 lớp chập
char weight_name[7][50]={
    "weights/conv0_W.txt",
    "weights/conv1_W.txt",
    "weights/conv2_W.txt",
    "weights/conv3_W.txt",
    "weights/conv4_W.txt",
    "weights/conv5_W.txt",
    "weights/conv6_W.txt"
};
//7 file bias của 7 lớp chập
char bias_name[7][50]={
    "weights/conv0_b.txt",
    "weights/conv1_b.txt",
    "weights/conv2_b.txt",
    "weights/conv3_b.txt",
    "weights/conv4_b.txt",
    "weights/conv5_b.txt",
    "weights/conv6_b.txt"
};
//ghi ảnh vào RAM
void write_img (const char *file_img,int insize,int nchannel,unsigned long *ddr_addr)
{
    int i,j,k;
    unsigned long wdata;
    int tmp0,tmp1;
    unsigned long wdata_addr;

```

```

wdata_addr = 0;
unsigned t = 240 * (cshape[0][2]+2)/2;
FILE *img;
img = fopen(file_img,"r");
if(img == NULL)
    printf("error open img");
for(k=0;k<t;k++)
{
    fscanf(img,"%d %d\n",&tmp1,&tmp0);
    wdata = (tmp1<<16)+tmp0;

    ddr_addr[wdata_addr] = wdata;
    wdata_addr++;
}
}
//ghi trọng số vào RAM
void write_weight(FILE* file_wei, int nchannel,int nkernel,unsigned long *ddr_addr)
{
    int i,j;
    int wdata;
    int tmp0,tmp1;
    unsigned long wdata_addr;
    unsigned long k;
    k = 9*16*nchannel*(nkernel/32);
    for(i=0;i<k;i++)
    {
        fscanf(file_wei,"%d %d\n",&tmp0,&tmp1);
        wdata = (tmp1 << 16)+tmp0;
        wdata_addr = i;
        ddr_addr[wdata_addr] = wdata;
    }
}
//ghi bias vào RAM
void write_bias (FILE *file_bs,int nkernel,unsigned long *ddr_addr)
{
    int i;
    int wdata;
    int tmp0,tmp1;
    unsigned long wdata_addr;
    for(i=0;i<nkernel/2;i++)
    {
        fscanf(file_bs,"%d %d\n",&tmp0,&tmp1);
        wdata = (tmp1<<16)+tmp0;
        wdata_addr = i;
        ddr_addr[wdata_addr ] = wdata;
    }
}
//cho phép bộ tăng tốc hoạt động
void write_start(unsigned long * low_cnn_addr)
{
    low_cnn_addr[0] = 1;
}
//ghi trạng thái các lớp
void write_layer_status(int conv,int pooling, int insize, int nkernel, int
nchannel,unsigned long * low_cnn_addr)
{
    int wdata;
    wdata = (conv <<31) + (pooling <<30) + (insize<<20) + (nkernel <<10) + nchannel;
    low_cnn_addr[2] = wdata;
}
//chạy một lớp chấp vào FPGA
void implement_a_layer(int pooling,int insize,int nkernel,int nchannel,unsigned long *
low_cnn_addr,unsigned long * ddr_addr,int layer,uint32_t rbase,uint32_t wbase)
{
    int a=0;
    write_layer_status(1,pooling,insize,nkernel,nchannel,low_cnn_addr);
    low_cnn_addr[4] = rbase;
    low_cnn_addr[5] =wbase;
    usleep(1);
    write_start(low_cnn_addr);
    while (!a)
    {

```

```

        a = low_cnn_addr[3];
        usleep(1);
    };
}
//khởi tạo vùng nhớ lưu trọng số và bias
void init_mem_conv(unsigned long * ddr_addr)
{
    FILE *w[NUM_LAYER];
    FILE *b[NUM_LAYER];
    for(int i=0;i<NUM_LAYER;i++)
    {
        w[i]=fopen(weight_name[i],"r");
        if(w[i] == NULL)
        {
            printf("err open wfile %d\n",i);
            return ;
        }

        b[i]=fopen(bias_name[i],"r");
        if(w[i] == NULL)
        {
            printf("err open bfile %d\n",i);
            return ;
        }
    }
    for(int i=0;i<NUM_LAYER;i++)
    {
        write_weight(w[i], cshape[i][1],cshape[i][0],ddr_addr + i*0x140000 +
0x200000);

        printf("done weight %d\n",i);
        write_bias (b[i],cshape[i][0],ddr_addr+i*0x100 + 0x1380000);
        printf("done bias %d\n",i);
    }
}
//hàm flatten
void flatten (int16_t *base_in, float *base_out)
{
    int i,j,k;
    unsigned long addr=0,addr_tmp;
    k = cshape[NUM_LAYER-1][2]*cshape[NUM_LAYER-1][2]*cshape[NUM_LAYER-1][0]/4;
    for (i=0;i<2048;i++)
    {
        base_out[i] = (float)base_in[i]/128;
    }
}
//thực hiện tính các lớp chập và flatten
void vgg16_convolution (unsigned long * low_cnn_addr,unsigned long * ddr_addr,float
*flatten_base)
{
    int i;
    unsigned long rbase =0x0 ,wbase=0x5000000;
    unsigned long tmp;
    for(i=0;i<NUM_LAYER;i++)
    {
        implement_a_layer(cshape[i][3],cshape[i][2],cshape[i][0],cshape[i][1],low_cnn_addr,
ddr_addr,i,rbase,wbase);
        tmp = wbase;
        wbase = rbase;
        rbase = tmp;
    }
    //*out_base = rbase;//4;
    flatten((int16_t*)(ddr_addr+rbase/4),flatten_base);
}

```

Nội dung file dense.h

```

#ifndef __DENSE_H__
#define __DENSE_H__
#include <stdio.h>

```

```

#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <omp.h>
#include <stdint.h>
#include <cmath>
extern float ***wd;
extern float **bd;
extern int numthreads;
extern int dshape[2][2];
extern const char weight_dense[3][50];
extern const char bias_dense[3][50];
extern int mem_block_dense_shape;
extern float *mem_block1_dense;
extern float *mem_block2_dense;
void reset_mem_block_dense(float *mem);
void init_memory();
void free_memory();
void read_weights();
void add_bias_and_relu_flatten(float *out, float *bs, int size, int relu);
void dense(float *in, float **weights, float *out, int sh_in, int sh_out);
void softmax(float *out, int sh_out);
int get_VGG16_predict(float *base_flatten, float *acc);
#endif

```

#### Nội dung file dense.cpp

```

#include "dense.h"
int dshape[2][2] = {
    { 2048, 128 },
    { 128, 7 }
};

float ***wd;
float **bd;
int numthreads = 2;
//2 file trọng số của 2 lớp dense
const char weight_dense[3][50] = { "weights/dense1_W.txt",
                                     "weights/dense2_W.txt" };
//2 file bias của 2 lớp dense
const char bias_dense[3][50] = { "weights/dense1_b.txt",
                                   "weights/dense2_b.txt" };

int mem_block_dense_shape = 128 * 4 * 4 ;
float *mem_block1_dense;
float *mem_block2_dense;
void reset_mem_block_dense(float *mem) {
    int i;
    for (i = 0; i < mem_block_dense_shape; i++) {
        mem[i] = 0.0;
    }
}
//khởi tạo vùng nhớ tính dense
void init_memory() {
    int i, j, k, l;
    // Init dense weights
    // Init dense weights
    wd = new float**[2]; //malloc(3 * sizeof(float**));
    bd = new float*[2]; //malloc(3 * sizeof(float*));
    for (l = 0; l < 2; l++) {
        wd[l] = new float*[dshape[l][1]]; //malloc(dshape[l][0] * sizeof(float*));
        for (i = 0; i < dshape[l][1]; i++) {
            wd[l][i] = new float [dshape[l][0]]; // malloc(dshape[l][0] *
sizeof(float));
        }
        bd[l] = new float[dshape[l][1]]; //malloc(dshape[l][1] * sizeof(float));
    }
    // Init mem blocks dense
    mem_block1_dense = new float
[mem_block_dense_shape]; //calloc(mem_block_dense_shape, sizeof(float));
    mem_block2_dense = new float
[mem_block_dense_shape]; //calloc(mem_block_dense_shape, sizeof(float));
}
//giải phóng vùng nhớ tính dense

```

```

void free_memory() {
    int i, j, k, l;
    // Free dense weights
    for (l = 0; l < 2; l++) {
        for (i = 0; i < dshape[l][1]; i++) {
            free(wd[l][i]);
        }
        free (wd[l]);
        free (bd[l]);
    }
    free(wd);
    free(bd);
    free(mem_block1_dense);
    free(mem_block2_dense);
}
//đọc trọng số và bias của dense
void read_weights() {
    float dval;
    float tval;
    int i, j, k, l, z;
    FILE *in_w[2];
    FILE *in_b[2];
    int total_lvls_read = 0;
    for(i=0;i<2;i++)
    {
        in_w[i] = fopen(weight_dense[i], "r");
        if (in_w[i] == NULL) {
            printf("File %d %s absent\n", i, weight_dense[i]);
            exit(1);
        }
    }
    for(i=0;i<2;i++)
    {
        in_b[i] = fopen(bias_dense[i], "r");
        if (in_b[i] == NULL) {
            printf("File %s absent\n", bias_dense[i]);
            exit(1);
        }
    }
    for (z = 0; z < 2; z++) {
        for (i = 0; i < dshape[z][1]; i++) {
            for (j = 0; j < dshape[z][0]; j++) {
                fscanf(in_w[z], "%f\n", &dval);
                wd[z][i][j] = dval;
            }
        }
        for (i = 0; i < dshape[z][1]; i++) {
            fscanf(in_b[z], "%f\n", &dval);
            bd[z][i] = dval;
        }
    }
    for(i=0;i<2;i++)
    {
        fclose(in_w[i]);
        fclose(in_b[i]);
    }
}
//hàm bias và relu
void add_bias_and_relu_flatten(float *out, float *bs, int size, int relu) {
    int i;
    for (i = 0; i < size; i++) {
        out[i] += bs[i];
        if (relu == 1) {
            if (out[i] < 0)
                out[i] = 0.0;
        }
    }
}
//Hàm tính 1 lớp dense trên CPU
void dense(float *in, float **weights, float *out, int sh_in, int sh_out) {
    int i, j;
    #pragma omp parallel for private(j) schedule(dynamic,1) num_threads(numthreads)
    for (i = 0; i < sh_out; i++) {
        float sum = 0.0;

```

```

        for (j = 0; j < sh_in; j++) {
            sum += in[j] * weights[i][j];
        }
        out[i] = sum;
    }
}

//hàm softmax
void softmax(float *out, int sh_out) {
    int i;
    float max_val, sum;
    max_val = out[0];
    for (i = 1; i < sh_out; i++) {
        if (out[i] > max_val)
            max_val = out[i];
    }
    sum = 0.0;
    for (i = 0; i < sh_out; i++) {
        out[i] = exp(out[i]);
        sum += out[i];
    }
    for (i = 0; i < sh_out; i++) {
        out[i] /= sum;
    }
}

//hàm tính các lớp dense
int get_VGG16_predict(float *base_flatten, float *acc) {
    int i, j=0;
    int level, cur_size;
    float max;
    reset_mem_block_dense(mem_block1_dense);
    reset_mem_block_dense(mem_block2_dense);
    level = 0;
    dense(base_flatten, wd[level], mem_block1_dense, dshape[level][0],
dshape[level][1]);
    add_bias_and_relu_flatten(mem_block1_dense, bd[level], dshape[level][1], 1);
    level = 1;
    dense(mem_block1_dense, wd[level], mem_block2_dense, dshape[level][0],
dshape[level][1]);
    add_bias_and_relu_flatten(mem_block2_dense, bd[level], dshape[level][1], 0);
    softmax(mem_block2_dense, dshape[level][1]);
    max = mem_block2_dense[0];
    for(i=1; i<dshape[level][1]; i++)
    {
        if(mem_block2_dense[i] > max)
        {
            max = mem_block2_dense[i] ;
            j = i;
        }
    }
    *acc = max;
    return j;
}

```

Nội dung file VideoFaceDetector.h, thư viện này chỉ dùng một số hàm cần thiết.

```

//tham khảo source: https://github.com/hrastnik/face\_detect\_n\_track
#pragma once

#include <opencv2/core.hpp>
#include <opencv2/highgui.hpp>
#include <opencv2/objdetect.hpp>

class VideoFaceDetector
{
public:
    VideoFaceDetector(const std::string cascadeFilePath, cv::VideoCapture
&videoCapture);
    ~VideoFaceDetector();

    cv::Point getFrameAndDetect(cv::Mat &frame);
    cv::Point operator>>(cv::Mat &frame);
    void setVideoCapture(cv::VideoCapture &videoCapture);
    cv::VideoCapture* videoCapture() const;

```

```

        void                setFaceCascade(const std::string cascadeFilePath);
        cv::CascadeClassifier* faceCascade() const;
        void                setResizedWidth(const int width);
        int                 resizedWidth() const;
        bool                isFaceFound() const;
        cv::Rect             face() const;
        cv::Point            facePosition() const;
        void                setTemplateMatchingMaxDuration(const double s);
        double               templateMatchingMaxDuration() const;

private:
        static const double  TICK_FREQUENCY;

        cv::VideoCapture*    m_videoCapture = NULL;
        cv::CascadeClassifier* m_faceCascade = NULL;
        std::vector<cv::Rect> m_allFaces;
        cv::Rect             m_trackedFace;
        cv::Rect             m_faceRoi;
        cv::Mat              m_faceTemplate;
        cv::Mat              m_matchingResult;
        bool                 m_templateMatchingRunning = false;
        int64                m_templateMatchingStartTime = 0;
        int64                m_templateMatchingCurrentTime = 0;
        bool                 m_foundFace = false;
        double               m_scale;
        int                  m_resizedWidth = 320;
        cv::Point            m_facePosition;
        double               m_templateMatchingMaxDuration = 3;

        cv::Rect             doubleRectSize(const cv::Rect &inputRect, const cv::Rect &frameSize)
const;
        cv::Rect             biggestFace(std::vector<cv::Rect> &faces) const;
        cv::Point            centerOfRect(const cv::Rect &rect) const;
        cv::Mat              getFaceTemplate(const cv::Mat &frame, cv::Rect face);
        void                 detectFaceAllSizes(const cv::Mat &frame);
        void                 detectFaceAroundRoi(const cv::Mat &frame);
        void                 detectFacesTemplateMatching(const cv::Mat &frame);
};

```

#### Nội dung file VideoFaceDetector.cpp

```

#include "VideoFaceDetector.h"
#include <iostream>
#include <opencv2/imgproc.hpp>

const double VideoFaceDetector::TICK_FREQUENCY = cv::getTickFrequency();

VideoFaceDetector::VideoFaceDetector(const std::string cascadeFilePath, cv::VideoCapture
&videoCapture)
{
    setFaceCascade(cascadeFilePath);
    setVideoCapture(videoCapture);
}

void VideoFaceDetector::setVideoCapture(cv::VideoCapture &videoCapture)
{
    m_videoCapture = &videoCapture;
}

cv::VideoCapture *VideoFaceDetector::videoCapture() const
{
    return m_videoCapture;
}

void VideoFaceDetector::setFaceCascade(const std::string cascadeFilePath)
{
    if (m_faceCascade == NULL) {
        m_faceCascade = new cv::CascadeClassifier(cascadeFilePath);
    }
    else {
        m_faceCascade->load(cascadeFilePath);
    }
}

```

```
        if (m_faceCascade->empty()) {
            std::cerr << "Error creating cascade classifier. Make sure the file" <<
std::endl
                << cascadeFilePath << " exists." << std::endl;
        }
    }

cv::CascadeClassifier *VideoFaceDetector::faceCascade() const
{
    return m_faceCascade;
}

void VideoFaceDetector::setResizedWidth(const int width)
{
    m_resizedWidth = std::max(width, 1);
}

int VideoFaceDetector::resizedWidth() const
{
    return m_resizedWidth;
}

bool VideoFaceDetector::isFaceFound() const
{
    return m_foundFace;
}

cv::Rect VideoFaceDetector::face() const
{
    cv::Rect faceRect = m_trackedFace;
    faceRect.x = (int)(faceRect.x / m_scale);
    faceRect.y = (int)(faceRect.y / m_scale);
    faceRect.width = (int)(faceRect.width / m_scale);
    faceRect.height = (int)(faceRect.height / m_scale);
    return faceRect;
}

cv::Point VideoFaceDetector::facePosition() const
{
    cv::Point facePos;
    facePos.x = (int)(m_facePosition.x / m_scale);
    facePos.y = (int)(m_facePosition.y / m_scale);
    return facePos;
}

void VideoFaceDetector::setTemplateMatchingMaxDuration(const double s)
{
    m_templateMatchingMaxDuration = s;
}

double VideoFaceDetector::templateMatchingMaxDuration() const
{
    return m_templateMatchingMaxDuration;
}

VideoFaceDetector::~VideoFaceDetector()
{
    if (m_faceCascade != NULL) {
        delete m_faceCascade;
    }
}

cv::Rect VideoFaceDetector::doubleRectSize(const cv::Rect &inputRect, const cv::Rect
&frameSize) const
{
    cv::Rect outputRect;
    // Double rect size
    outputRect.width = inputRect.width * 2;
    outputRect.height = inputRect.height * 2;

    // Center rect around original center
    outputRect.x = inputRect.x - inputRect.width / 2;
    outputRect.y = inputRect.y - inputRect.height / 2;
}
```

```

        // Handle edge cases
        if (outputRect.x < frameSize.x) {
            outputRect.width += outputRect.x;
            outputRect.x = frameSize.x;
        }
        if (outputRect.y < frameSize.y) {
            outputRect.height += outputRect.y;
            outputRect.y = frameSize.y;
        }

        if (outputRect.x + outputRect.width > frameSize.width) {
            outputRect.width = frameSize.width - outputRect.x;
        }
        if (outputRect.y + outputRect.height > frameSize.height) {
            outputRect.height = frameSize.height - outputRect.y;
        }

        return outputRect;
    }

    cv::Point VideoFaceDetector::centerOfRect(const cv::Rect &rect) const
    {
        return cv::Point(rect.x + rect.width / 2, rect.y + rect.height / 2);
    }

    cv::Rect VideoFaceDetector::biggestFace(std::vector<cv::Rect> &faces) const
    {
        assert(!faces.empty());

        cv::Rect *biggest = &faces[0];
        for (auto &face : faces) {
            if (face.area() < biggest->area())
                biggest = &face;
        }
        return *biggest;
    }

    /*
    * Face template is small patch in the middle of detected face.
    */
    cv::Mat VideoFaceDetector::getFaceTemplate(const cv::Mat &frame, cv::Rect face)
    {
        face.x += face.width / 4;
        face.y += face.height / 4;
        face.width /= 2;
        face.height /= 2;

        cv::Mat faceTemplate = frame(face).clone();
        return faceTemplate;
    }

    void VideoFaceDetector::detectFaceAllSizes(const cv::Mat &frame)
    {
        // Minimum face size is 1/5th of screen height
        // Maximum face size is 2/3rds of screen height
        m_faceCascade->detectMultiScale(frame, m_allFaces, 1.1, 3, 0,
            cv::Size(frame.rows / 5, frame.rows / 5),
            cv::Size(frame.rows * 2 / 3, frame.rows * 2 / 3));

        if (m_allFaces.empty()) return;

        m_foundFace = true;

        // Locate biggest face
        m_trackedFace = biggestFace(m_allFaces);

        // Copy face template
        m_faceTemplate = getFaceTemplate(frame, m_trackedFace);

        // Calculate roi
        m_faceRoi = doubleRectSize(m_trackedFace, cv::Rect(0, 0, frame.cols,
            frame.rows));

        // Update face position
    }

```

```

        m_facePosition = centerOfRect(m_trackedFace);
    }

void VideoFaceDetector::detectFaceAroundRoi(const cv::Mat &frame)
{
    // Detect faces sized +/-20% off biggest face in previous search
    m_faceCascade->detectMultiScale(frame(m_faceRoi), m_allFaces, 1.1, 3, 0,
        cv::Size(m_trackedFace.width * 8 / 10, m_trackedFace.height * 8 / 10),
        cv::Size(m_trackedFace.width * 12 / 10, m_trackedFace.width * 12 / 10));

    if (m_allFaces.empty())
    {
        // Activate template matching if not already started and start timer
        m_templateMatchingRunning = true;
        if (m_templateMatchingStartTime == 0)
            m_templateMatchingStartTime = cv::getTickCount();
        return;
    }

    // Turn off template matching if running and reset timer
    m_templateMatchingRunning = false;
    m_templateMatchingCurrentTime = m_templateMatchingStartTime = 0;

    // Get detected face
    m_trackedFace = biggestFace(m_allFaces);

    // Add roi offset to face
    m_trackedFace.x += m_faceRoi.x;
    m_trackedFace.y += m_faceRoi.y;

    // Get face template
    m_faceTemplate = getFaceTemplate(frame, m_trackedFace);

    // Calculate roi
    m_faceRoi = doubleRectSize(m_trackedFace, cv::Rect(0, 0, frame.cols,
frame.rows));

    // Update face position
    m_facePosition = centerOfRect(m_trackedFace);
}

void VideoFaceDetector::detectFacesTemplateMatching(const cv::Mat &frame)
{
    // Calculate duration of template matching
    m_templateMatchingCurrentTime = cv::getTickCount();
    double duration = (double)(m_templateMatchingCurrentTime -
m_templateMatchingStartTime) / TICK_FREQUENCY;

    // If template matching lasts for more than 2 seconds face is possibly lost
    // so disable it and redetect using cascades
    if (duration > m_templateMatchingMaxDuration) {
        m_foundFace = false;
        m_templateMatchingRunning = false;
        m_templateMatchingStartTime = m_templateMatchingCurrentTime = 0;
        m_facePosition.x = m_facePosition.y = 0;
        m_trackedFace.x = m_trackedFace.y = m_trackedFace.width =
m_trackedFace.height = 0;
        return;
    }

    // Edge case when face exits frame while
    if (m_faceTemplate.rows * m_faceTemplate.cols == 0 || m_faceTemplate.rows <= 1 ||
m_faceTemplate.cols <= 1) {
        m_foundFace = false;
        m_templateMatchingRunning = false;
        m_templateMatchingStartTime = m_templateMatchingCurrentTime = 0;
        m_facePosition.x = m_facePosition.y = 0;
        m_trackedFace.x = m_trackedFace.y = m_trackedFace.width =
m_trackedFace.height = 0;
        return;
    }

    // Template matching with last known face

```

```

        //cv::matchTemplate(frame(m_faceRoi), m_faceTemplate, m_matchingResult,
CV_TM_CCORR);
        cv::matchTemplate(frame(m_faceRoi), m_faceTemplate, m_matchingResult,
CV_TM_SQDIFF_NORMED);
        cv::normalize(m_matchingResult, m_matchingResult, 0, 1, cv::NORM_MINMAX, -1,
cv::Mat());
        double min, max;
        cv::Point minLoc, maxLoc;
        cv::minMaxLoc(m_matchingResult, &min, &max, &minLoc, &maxLoc);

        // Add roi offset to face position
        minLoc.x += m_faceRoi.x;
        minLoc.y += m_faceRoi.y;

        // Get detected face
        //m_trackedFace = cv::Rect(maxLoc.x, maxLoc.y, m_trackedFace.width,
m_trackedFace.height);
        m_trackedFace = cv::Rect(minLoc.x, minLoc.y, m_faceTemplate.cols,
m_faceTemplate.rows);
        m_trackedFace = doubleRectSize(m_trackedFace, cv::Rect(0, 0, frame.cols,
frame.rows));

        // Get new face template
        m_faceTemplate = getFaceTemplate(frame, m_trackedFace);

        // Calculate face roi
        m_faceRoi = doubleRectSize(m_trackedFace, cv::Rect(0, 0, frame.cols,
frame.rows));

        // Update face position
        m_facePosition = centerOfRect(m_trackedFace);
    }

cv::Point VideoFaceDetector::getFrameAndDetect(cv::Mat &frame)
{
    *m_videoCapture >> frame;

    // Downscale frame to m_resizedWidth width - keep aspect ratio
    m_scale = (double)std::min(m_resizedWidth, frame.cols) / frame.cols;
    cv::Size resizedFrameSize = cv::Size((int)(m_scale*frame.cols),
(int)(m_scale*frame.rows));

    cv::Mat resizedFrame;
    cv::resize(frame, resizedFrame, resizedFrameSize);

    if (!m_foundFace)
        detectFaceAllSizes(resizedFrame); // Detect using cascades over whole
image
    else {
        detectFaceAroundRoi(resizedFrame); // Detect using cascades only in ROI
        if (m_templateMatchingRunning) {
            detectFacesTemplateMatching(resizedFrame); // Detect using
template matching
        }
    }

    return m_facePosition;
}

cv::Point VideoFaceDetector::operator>>(cv::Mat &frame)
{
    return this->getFrameAndDetect(frame);
}

```